Algorithmics

(CT065-3-3-ALG)

Group Assignment 1

Lecturer Name : Ms. Maznah binti Mohd Zin

Hand out Date : 7 March 2026

Hand in Date : 10 May 2026

Intake Code : APU3F2601CS(DA) / APD3F2601CS(DA)

Group No : 11

Group Members

Student ID	Student Name
TP068983	Chan Chun Ming
TP068580	Eu Jun Hong
TP068495	Lim Beng Rhui (*Group Leader*)
TP067678	Ng Xuan Jack

Table of Contents

Workload Matrix ····· 3
Introduction ····· 4
Part A: Data Structure and Algorithm Representation ····· 5
Data Structure: Simple Directed Weighted Graph ····· 5
Types of Graphs ····· 5
Representation of Graphs ····· 6
Advantages and Disadvantages ····· 7
Algorithm Representation: Bellman-Ford ····· 8
Algorithm Representation: Floyd-Warshall ····· 9
Part B: Algorithm Complexity Analysis ····· 10
Algorithm 1: Bellman-Ford ····· 10
Algorithm Correctness ····· 11
Worst Case Analysis: Step-by-Step Trace ····· 14
Worst Case Analysis: Time Complexity ····· 16
Best Case Analysis ····· 18
Algorithm 2: Floyd-Warshall ····· 19
Algorithm Correctness ····· 20
Worst Case Analysis: Step-by-Step Trace ····· 24
Worst Case Analysis: Time Complexity ····· 26
Best Case Analysis ····· 28
Part C: Comparison and Evaluation between Algorithms ····· 29
Performance Comparison ····· 29
Algorithm Use Case ····· 32
Improvement Suggestions ····· 33
Conclusion ····· 34
References ····· 35
Appendix A. Code Implementation of Graph in Java ····· 36
Appendix B. Trace Table for Worst Case Scenario of Bellman-Ford Algorithm ····· 38
Appendix C. Trace Table for Best Case Scenario of Bellman-Ford Algorithm ····· 42
Appendix D. Trace Table for Worst Case Scenario of Floyd-Warshall Algorithm ····· 44
Appendix E. Trace Table for Best Case Scenario of Floyd-Warshall Algorithm ····· 53
Appendix F. Modified Code Snippet for Tracking Execution Time ····· 57

Workload Matrix

Task		Contribution Percentage			
		Chan Chun Ming (TP068983)	**Eu Jun Hong (TP068580)**	**Lim Beng Rhui (TP068495)**	**Ng Xuan Jack (TP067678)**
Introduction		-	-	-	100%
Part A	Data Structure Description	-	75%	25%	-
	Bellman-Ford: Algorithm Representation	-	-	-	100%
	Floyd-Warshall: Algorithm Representation	-	100%	-	-
Part B	Bellman-Ford: Worst-Case Step by Step Trace	100%	-	-	-
	Bellman-Ford: Worst-Case Complexity Analysis	100%	-	-	-
	Floyd-Warshall: Worst-Case Step by Step Trace	-	-	100%	-
	Floyd-Warshall: Worst-Case Complexity Analysis	-	-	100%	-
Part C	Comparison and Evaluation between Algorithms	50%	-	50%	-
Conclusion		-	-	100%	-
Overall Contribution		**30%**	**20%**	**30%**	**20%**

Introduction

With the advancement of technology in the current era, the rapid development of urban areas has transformed cities into complicated mazes, where people have to traverse through complex routes to reach their destinations, making efficient navigation a necessity to support human's daily activities. Hence, this report looks into the algorithms behind navigational maps by comparing two algorithms, namely Bellman-Ford and Floyd-Warshall. By comparing their performances and complexities, the report intends to outline their strengths and weaknesses to gain insight on their practical integration within real-life applications.

Part A: Data Structure and Algorithm Representation

Simple directed weighted graph is selected as the data structure of this report, as both Bellman-Ford and Floyd-Warshall algorithm relies on weighted graphs to determine the shortest distance between two nodes.

Data Structure: Simple Directed Weighted Graph

A graph is a non-linear data structure that represents a collection of nodes and edges (Maung, 2025). Mathematically, graphs are defined as:

$$G = (V, E)$$

where V represents the set of vertices, or nodes, that stores data entities, and E represents the set of edges that joins different vertices together. The connection between nodes allows graphs to represent intricate relationships between different data points, allowing it to be applied in a wide range of fields, including transportation networks, social relationships, and electrical circuit connections (Bharathi & Kaviya, 2025).

Types of Graphs

Graphs are divided into different categories, where they can be broken down into:

- **Simple and multigraph**
  In a simple graph, any two nodes are connected by at most one edge to ensure clean relationship. Multigraphs, in contrast, allow multiple edges to exist between nodes.

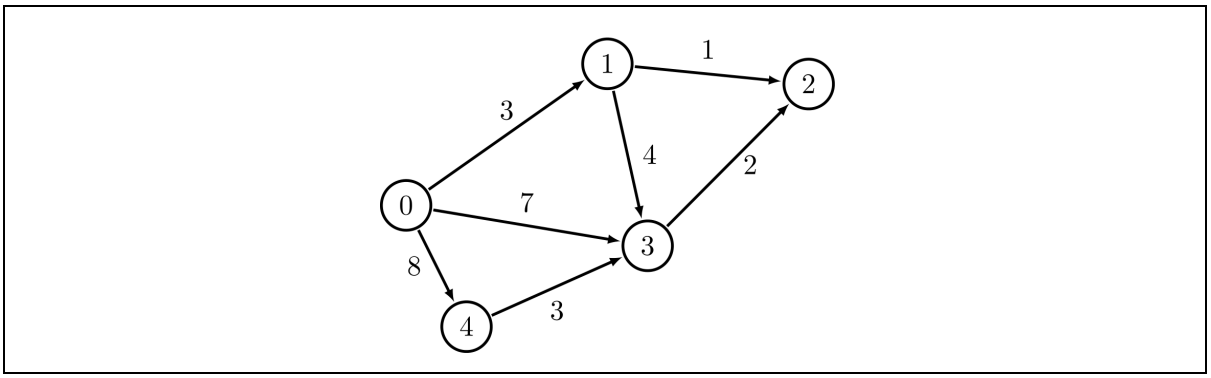
- **Directed and undirected**
  The edges in a directed graph have a specific direction that points from a starting node to a destination node, while no direction exists in the edges of undirected graphs, indicating mutual relationship between two nodes in undirected graphs.

- **Weighted and unweighted**
  A weighted graph has a numerical value assigned to all its edges, representing information such as cost or distance, unlike the edges in unweighted graphs that does not have values.

An example of a graph fulfilling the characteristics of a simple directed weighted graph is shown in Figure 1, where similar graphs will be used to demonstrate both Bellman-Ford and Floyd-Warshall algorithms.

Figure 1

An Example of Simple Directed Weighted Graph



Note. From *Weighted graph*, by Hyperskill, n.d. (https://hyperskill.org/learn/step/5645).

Representation of Graphs

In a programming context, graphs are usually stored in two methods (Nayyar, 2024):

- **Adjacency list**

  In adjacency lists, graphs are stored as a collection of lists, where each list represents a node, and the elements in the list represent the adjacent nodes of the corresponding vertex. Adjacency lists are often more memory efficient when storing sparse graphs.

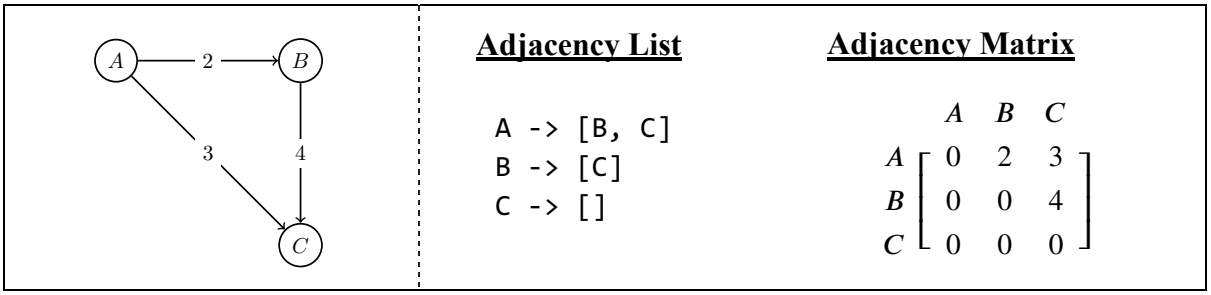
- **Adjacency matrix**

  An adjacency matrix is a 2D array in which its rows and columns represent the nodes. If an edge exists between node *i* and *j*, the weight of the edge is recorded in `matrix[i][j]`. Adjacency matrix is preferred when storing dense graph as it supports efficient lookup.

An example of graph represented in adjacency list and matrix is shown in Figure 2, and its code implementation in Java is shown in Appendix A.

Figure 2

Graph Represented in Adjacency List and Adjacency Matrix



Advantages and Disadvantages

According to NxtWave (2025), some of the advantages of using graphs to store data include:

- **Highly versatile**

  Graphs can be used to represent different data types, such as trees and matrices, and capture the relationship between different data points in a flexible and efficient manner. This allows graphs to be widely implemented in real-world applications like social and communication networks.

- **Efficient data query and retrieval**

  As graphs support different searching algorithms, such as breadth-first and depth-first search, it is efficient to traverse within graphs to find and return values from nodes. This allows graphs to improve performance on systems like recommendation systems.

On the other hand, the challenges of using graphs include:

- **Space and time complexity**

  As the size of the graph becomes larger, the memory required to represent graph becomes significantly high. Also, performing traversal on large graphs is time-consuming and computationally expensive.

- **Visualization**

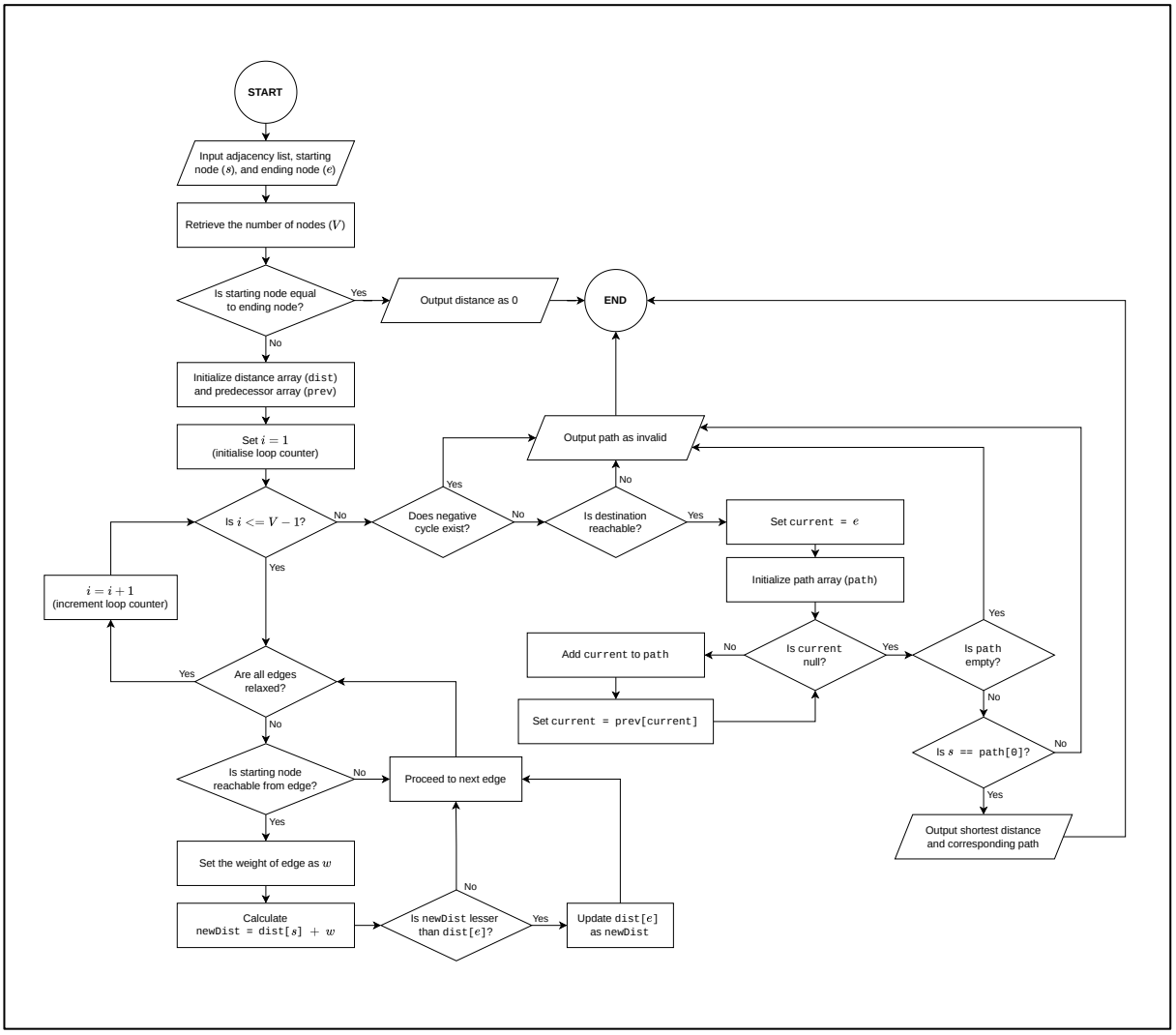
  It is also difficult to visualize relationships between nodes when the graph becomes too large and complex, which poses challenges for viewers to extract valuable information from the graph while adding unnecessary noise during interpretation.

Algorithm Representation: Bellman-Ford

As shown in Figure 3, the Bellman-Ford algorithm first initialises the distance array by setting the self-distance of the starting node as zero, and the distance to all other vertices as infinity, along with a predecessor array for path tracking. Since the longest possible shortest path in a graph with V vertices can have at most $V - 1$ edges, the algorithm undergoes $V - 1$ iterations by traversing all edges in each iteration to perform edge relaxation and look for a shorter path. The distance and predecessor arrays are then updated if a shorter path is found. Once the iteration process completes, the algorithm checks for the existence of negative cycles to avoid infinite loops that invalidate the shortest path, then lastly backtracks the predecessor array to reconstruct the shortest path from the destination node.

Figure 3

Flowchart of the Bellman-Ford Algorithm

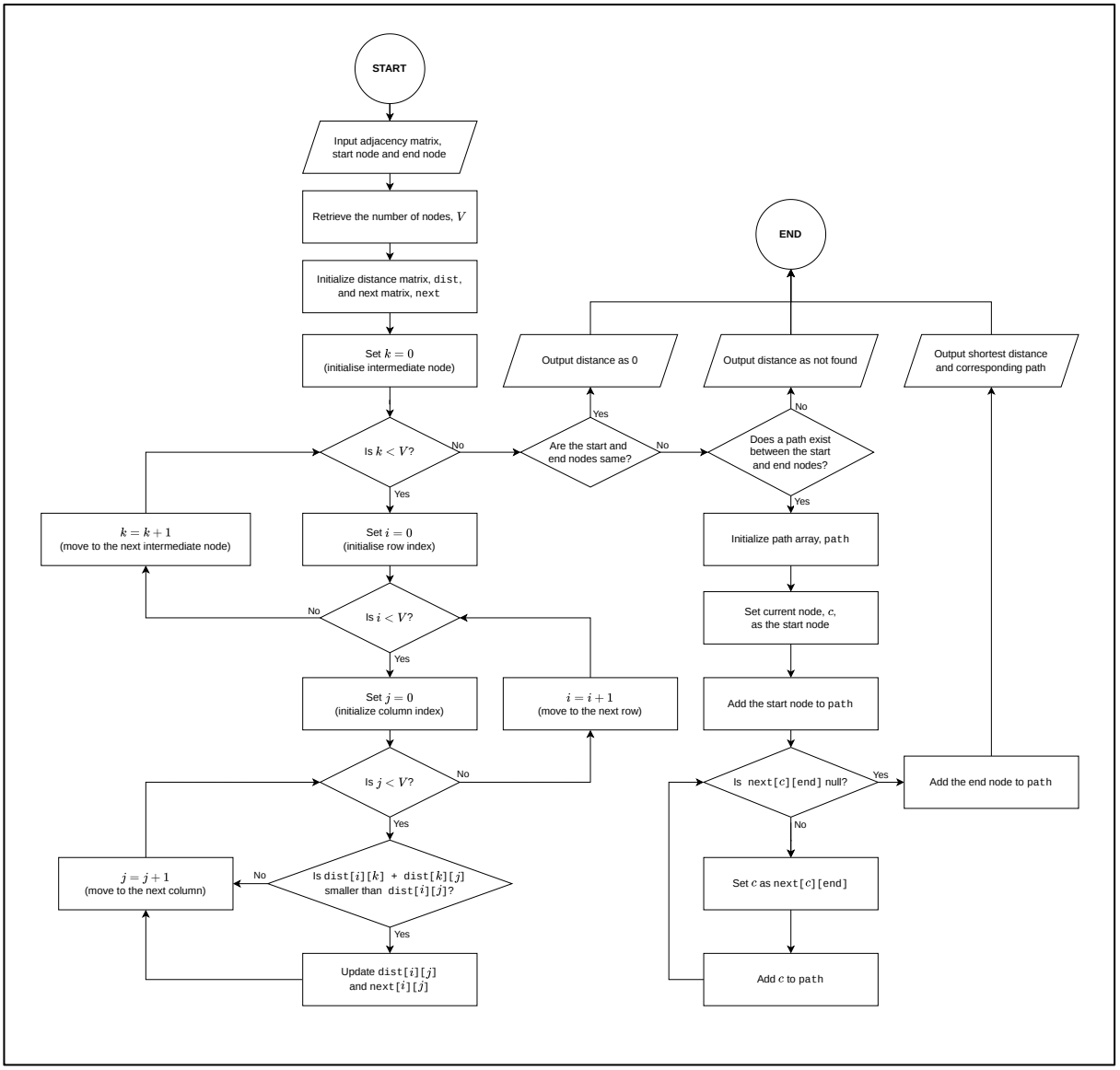


Algorithm Representation: Floyd-Warshall

The flowchart in Figure 4 depicts the tasks involved in the Floyd-Warshall algorithm. The algorithm first receives the adjacency matrix (used as the initial distance matrix), starting node and ending node, then initializes a next matrix to keep track of the intermediary nodes. After that, the algorithm executes a triple-nested loop, where the distance of each pair node is compared with the distance when an intermediary node is involved, and both the distance and next matrices are updated to if a shorter distance is detected. The algorithm then proceeds with checking for special cases, such as self-distance and invalid paths, and lastly, backtracks the next matrix to retrieve relevant intermediary nodes and returns the shortest distance and path.

Figure 4

Flowchart of the Floyd-Warshall Algorithm



Part B: Algorithm Complexity Analysis

Algorithm 1: Bellman-Ford

Figure 5 shows the Java code snippet that demonstrates the Bellman-Ford algorithm.

Figure 5

Code Snippet of Bellman-Ford Algorithm for Time and Space Complexity Analysis

```java
public class Bellman_Ford {
    public static String getShortestPath(Graph graph, Graph.Vertex start, Graph.Vertex destination) {

        // Pre-checking
        ArrayList<Graph.Vertex> vertices = graph.getSortedVertices();
        int verticesCount = vertices.size();
        if (start.equals(destination)) return "Distance from " + start + " to " + destination + " is: 0";
        Map<Graph.Vertex, Integer> indexOf = new HashMap<>();
        for (int i = 0; i < verticesCount; i++) indexOf.put(vertices.get(i), i);
        Integer startIdx = indexOf.get(start);
        Integer endIdx = indexOf.get(destination);
        if (startIdx == null || endIdx == null) return "Error. No path is found.";

        // Initialization
        double[] dist = new double[verticesCount];
        Arrays.fill(dist, Double.POSITIVE_INFINITY);
        int[] prev = new int[verticesCount];
        Arrays.fill(prev, -1);
        dist[startIdx] = 0.0;
        ArrayList<EdgeIndex> edges = new ArrayList<>();
        for (Graph.Edge edge : graph.getEdges()) {
           Integer startNode = indexOf.get(edge.startNode);
           Integer endNode = indexOf.get(edge.endNode);
           if (startNode != null && endNode != null) {
              edges.add(new EdgeIndex(startNode, endNode, edge.weight));
        }  }

        // Bellman-Ford algorithm execution
        for (int i = 0; i < verticesCount - 1; i++) {
           for (EdgeIndex edge : edges) {
              if (dist[edge.startIndex] == Double.POSITIVE_INFINITY) continue;
              double newDist = dist[edge.startIndex] + edge.weight;
              if (newDist < dist[edge.endIndex]) {
                 dist[edge.endIndex] = newDist;
                 prev[edge.endIndex] = edge.startIndex;
        }  }  }

        // Check for negative cycle and path validity
        for (EdgeIndex e : edges) {
           if (dist[e.startIndex] == Double.POSITIVE_INFINITY) continue;
           if (dist[e.startIndex] + e.weight < dist[e.endIndex]) {
              return "Negative cycle found from vertex " + start + ". This path is invalid.";
        }  }
        double shortest = dist[endIdx];
        if (shortest == Double.POSITIVE_INFINITY) {
           return "Distance from " + start + " to " + destination + " is: Not found";
        }

        // Backtracking for shortest path
        ArrayList<String> path = new ArrayList<>();
        int current = endIdx;
        while (current != -1) {
           path.addFirst(vertices.get(current).label());
           if (current == startIdx) break;
           current = prev[current];
        }
        if (path.isEmpty() || !path.getFirst().equals(start.label())) return "Error. No path is found.";
        return "Distance from " + start + " to " + destination + " is: " + shortest +
              "\nThe shortest path is: " + String.join(" -> ", path);
    }
}
```

Algorithm Correctness

To ensure the Bellman-Ford algorithm provides accurate results when handling different types of graphs, two testing methods have been implemented, including empirical testing and invariant testing, to validate the correctness of the algorithm under different circumstances (Meegle, 2026).

- **Empirical testing**

  The function to perform empirical testing for the Bellman-Ford algorithm is shown in Figure 6, where four different scenarios that might be encountered during algorithm execution were tested, including normal case, missing path, negative edge, and negative cycle. The output in Figure 7 shows that the algorithm has successfully passed all the test cases with the actual output tallying with the expected results.

Figure 6

Code for Empirical Testing of Bellman-Ford Algorithm

```
public static void runBellmanEmpirical() {
    System.out.println("Running Empirical Tests...\n");
    int pass = 0;
    int total = 4;

    // Test 1: Normal case, shortest path A -> C -> B = 3
    Graph g1 = new Graph("[(A,B,4); (A,C,2); (C,B,1)]");
    Graph.Vertex e1Start = Graph.Vertex.find("A");
    Graph.Vertex e1End = Graph.Vertex.find("B");
    String result1 = (e1Start == null || e1End == null)
            ? "Error. Test setup failed."
            : Bellman_Ford.getShortestPath(g1, e1Start, e1End);
    boolean test1 = result1.contains("is: 3.0") || result1.contains("is: 3");
    System.out.println("Test 1 Result:\n" + result1);
    System.out.println("Expected: Distance = 3");
    System.out.println("PASS = " + test1 + "\n");
    if (test1) pass++;

    // Test 2: No path from X to Y
    Graph g2 = new Graph("[(X,Z,1); (W,Y,1)]");
    Graph.Vertex e2Start = Graph.Vertex.find("X");
    Graph.Vertex e2End = Graph.Vertex.find("Y");
    String result2 = (e2Start == null || e2End == null)
            ? "Error. Test setup failed."
            : Bellman_Ford.getShortestPath(g2, e2Start, e2End);
    boolean test2 = result2.contains("Not found") || result2.contains("No path");
    System.out.println("Test 2 Result:\n" + result2);
    System.out.println("Expected: Not found");
    System.out.println("PASS = " + test2 + "\n");
    if (test2) pass++;

    // Test 3: Negative edge is supported, R -> Q -> R = 1
    Graph g3 = new Graph("[(P,Q,-2); (Q,R,3)]");
    Graph.Vertex e3Start = Graph.Vertex.find("P");
    Graph.Vertex e3End = Graph.Vertex.find("R");
    String result3 = (e3Start == null || e3End == null)
            ? "Error. Test setup failed."
            : Bellman_Ford.getShortestPath(g3, e3Start, e3End);
    boolean test3 = result3.contains("is: 1.0") || result3.contains("is: 1");
    System.out.println("Test 3 Result:\n" + result3);
    System.out.println("Expected: Distance = 1");
    System.out.println("PASS = " + test3 + "\n");
    if (test3) pass++;
```

```
    // Test 4: Negative cycle should be detected
    Graph g4 = new Graph("[(U,V,1); (V,U,-2)]");
    Graph.Vertex e4Start = Graph.Vertex.find("U");
    Graph.Vertex e4End = Graph.Vertex.find("V");
    String result4 = (e4Start == null || e4End == null)
            ? "Error. Test setup failed."
            : Bellman_Ford.getShortestPath(g4, e4Start, e4End);
    boolean test4 = result4.toLowerCase().contains("negative cycle");
    System.out.println("Test 4 Result:\n" + result4);
    System.out.println("Expected: Negative cycle detected");
    System.out.println("PASS = " + test4 + "\n");
    if (test4) pass++;

    System.out.println("Empirical Tests Passed: " + pass + "/" + total + "\n");
}
```

Figure 7

Output for Empirical Testing of Bellman-Ford Algorithm

```
Running Empirical Tests...

Test 1 Result:
Distance from A to B is: 3.0
The shortest path is: A -> C -> B
Expected: Distance = 3
PASS = true

Test 2 Result:
Distance from X to Y is: Not found
Expected: Not found
PASS = true

Test 3 Result:
Distance from P to R is: 1.0
The shortest path is: P -> Q -> R
Expected: Distance = 1
PASS = true

Test 4 Result:
Negative cycle found from vertex U. This path is invalid.
Expected: Negative cycle detected
PASS = true

Empirical Tests Passed: 4/4


Process finished with exit code 0
```

- **Invariant testing**

  To implement invariant testing, predefined rules are set up, usually within the outer loop of the algorithm, to ensure that errors can be detected early before the algorithm moves to the next iteration (Cyfrin, n.d.). In the Bellman-Ford algorithm, three invariants are set, which includes:

  1. **Relaxation consistency**

     The code snippet in Figure 8 checks if the algorithm has successfully updated the shorter distance into the distance array. If the code detects any new distance that is larger than the existing shorter distance, assertions are raised to avoid inconsistency.

2. **Distance monotonicity**

   Before each inner iteration takes place, the distance array will be temporarily copied to allow comparison between the initial distance array with the outputted distance array. This is to ensure that the output distance will either be unchanged or decreased during each iteration, but not increase and yield inaccurate results.

3. **Unreachable vertex validation**

   The distance to a vertex shall remain infinity if it is unreachable. Hence, by implementing the code snippet in Figure 8, any incorrect indexing that illegally updates the distances of unreachable vertices is detected and prevented.

Based on the output in Figure 8, since the algorithm does not run into any assertion errors during its execution, the invariant testing is passed, and hence, the correctness of the Bellman-Ford algorithm is supported.

Figure 8

Code Snippet and Output for Invariant Testing of Bellman-Ford Algorithm

```
// Invariant 1: relaxation consistency check
if (invariant) {
    for (EdgeIndex e : edges) {
       if (dist[e.startIndex] != Double.POSITIVE_INFINITY) {
          assert (dist[e.endIndex] > dist[e.startIndex] + e.weight): "Invariant violated at edge: " + e;
}   }  }

// Invariant 2: distances should never increase after relaxation
if (invariant) {
    for (int j = 0; j < dist.length; j++) {
       assert (dist[j] > prevDist[j]): "Invariant violated: distance increased!";
}   }

// Invariant 3: if a node is unreachable in the previous iteration, its distance should remain ∞
if (invariant) {
    for (int j = 0; j < dist.length; j++) {
       assert (prev[j] == -1 && j != startIdx && dist[j] != Double.POSITIVE_INFINITY):
          "Invariant warning: unexpected reachable node";
}   }
```

```
--- Bellman-Ford: Graph 1: ---
Distance from A to D is: 5.0
The shortest path is: A -> B -> D

--- Bellman-Ford: Graph 2: ---
Distance from A to E is: 4.0
The shortest path is: A -> B -> E

--- Bellman-Ford: Graph 3: ---
Distance from A to F is: 9.0
The shortest path is: A -> B -> D -> E -> F

--- Bellman-Ford: Graph 4: ---
Distance from A to F is: 6.0
The shortest path is: A -> D -> F

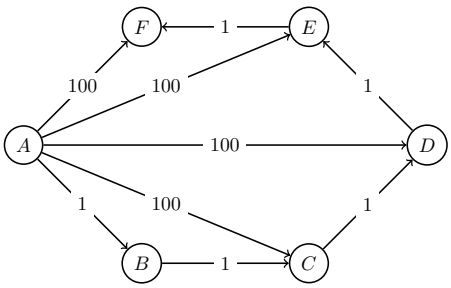

Process finished with exit code 0
```

Worst Case Analysis: Step-by-Step Trace

Since Bellman-Ford algorithm will go through every edge in the graph multiple times to check whether a shorter path is found, the fixed process does not yield worst case in terms of time complexity. Hence, the worst case focuses on demonstrating the case when the algorithm has to pass through the most number of vertex and update its known path in every iteration.

Figure 9

Graph for Bellman-Ford's Worst-Case Analysis Demonstration



Adjacency List Representation:

```
A -> [(A, B, 1), (A, C, 100), (A, D, 100),
      (A, E, 100), (A, F, 100)]
B -> [(B, C, 1)]
C -> [(C, D, 1)]
D -> [(D, E, 1)]
E -> [(E, F, 1)]
F -> []
```

The graph used for worst-case analysis demonstration is shown in Figure 9, containing several characteristics that cause the Bellman-Ford algorithm to execute all iterations, including:

- **Linear chain structure**

  For each iteration, the distance of each vertex can only be updated once, as the algorithm follows a sequential relaxation process that makes it unable to compute the distance of next vertex without the previous vertex, making edge relaxation operations and distance updates to take place step-by-step during algorithm execution (Jukna & Schnitger, 2016).

- **Non-optimal direct edges**

  The direct edges initialises distances that are reachable but not the shortest, forcing the algorithm to improve all these bad estimates by performing edge relaxation step-by-step and frequently updating the distance and predecessor arrays.

- **Sub-optimal edge order**

  The code in Figure 5 processes each edge based on their sequential order. If edges are processed before their source vertices are updated, edge relaxation will fail and cause the shortest path to only be discovered in later iterations, hence causing the storing order of edges to impact the algorithm's overall performance.

The trace table in Appendix B shows the step-by-step update of the distance and predecessor arrays when the Bellman-Ford algorithm is executed on the worst-case. It is observed that the algorithm exhibits several behaviours:

- **Edges are skipped in early iterations**

  As the Bellman-Ford algorithm relaxes edges in a non-topological order, it is possible for the algorithm to skip edges that contribute to the shortest path, due to the reason where their respective starting vertices have not yet been reached by the algorithm. Such a behaviour is observed from the trace table, where taking the edge `(E, F, 1)` as example,

Stage	`dist` array	`prev` array	Checking	Status
Initiation	[0, ∞, ∞, ∞, ∞, ∞]	[-1, -1, -1, -1, -1, -1]	-	-
Iteration 1	[0, ∞, ∞, ∞, ∞, ∞]	[-1, -1, -1, -1, -1, -1]	∞	Skipped
Iteration 2	[0, 1, 100, 100, 100, 100]	[-1, 0, 0, 0, 0, 0]	100 + 1 > 100	Rejected
Iteration 3	[0, 1, 2, 3, 100, 100]	[-1, 0, 1, 2, 0, 0]	100 + 1 > 100	Rejected
Iteration 4	[0, 1, 2, 3, 4, 100] -> [0, 1, 2, 3, 4, 5]	[-1, 0, 1, 2, 3, 0] -> [-1, 0, 1, 2, 3, 4]	4 + 1 < 100	Updated

  the edge `(E, F, 1)`, despite contributing to the shortest path, only updates the distance and predecessor arrays in its fourth iteration, due to the reason that the optimal path to its source node, `E`, is not detected until the third iteration, thereby increasing unnecessary operations that prevents the shortest distance from being identified efficiently.

- **Multiple updates towards the same element on the distance array**

  During algorithm execution, it is observed that the input graph contains many distractor edges that leads to additional array updates in every iteration. Taking vertex `D` as example,

  `dist[D] = ∞` ⟶ `dist[D] = 100` ⟶ `dist[D] = 3`

  its shortest distance from A is updated in several iterations, where it is first updated to 100, which is the incorrect value, before updated to 3 in subsequent iterations. Such a scenario increases array updates without contributing to shortest path identification.

Therefore, due to the non-topological order of edges, as well as the multiple updates towards the distance and predecessor array, the graph in Figure 9 demonstrates the Bellman-Ford algorithm that has a time complexity of $O(VE)$ but yields maximal computational costs.

Worst Case Analysis: Time Complexity

Based on the code snippet shown in Figure 5, the Bellman-Ford algorithm comprises a total of four processes, from data initialization to path construction.

1. Data initialization and preprocessing

```
double[] dist = new double[verticesCount];
Arrays.fill(dist, Double.POSITIVE_INFINITY);
int[] prev = new int[verticesCount];
Arrays.fill(prev, -1);
dist[startIdx] = 0.0;

ArrayList<EdgeIndex> edges = new ArrayList<>();
for (Graph.Edge edge : graph.getEdges()) {
    Integer startNode = indexOf.get(edge.startNode);
    Integer endNode = indexOf.get(edge.endNode);
    if (startNode != null && endNode != null) {
       edges.add(new EdgeIndex(startNode, endNode, edge.weight));
}   }
```

The initialization of two arrays, namely `dist` and `prev`, to store the shortest distance and the index of predecessor vertices respectively, takes a time complexity of $O(V)$ as the time taken to allocate memory and assign initial values depends on the number of vertices. Next, a `for`-loop is used to convert the edges of the graph into `EdgeIndex` objects to convert vertices into indices and allow array indexing during algorithm execution, which results in a time complexity of $O(E)$. This leads to an overall time complexity of:

$$O(V) + O(V) + O(V) + O(V) + O(E) = O(V) + O(E)$$

2. Execution of Bellman-Ford algorithm

```
for (int i = 0; i < verticesCount - 1; i++) {
    for (EdgeIndex edge : edges) {
       if (dist[edge.startIndex] == Double.POSITIVE_INFINITY) continue;
       double newDist = dist[edge.startIndex] + edge.weight;
       if (newDist < dist[edge.endIndex]) {
          dist[edge.endIndex] = newDist;
          prev[edge.endIndex] = edge.startIndex;
}   }  }
```

As the longest path between two nodes has $V - 1$ number of edges, the Bellman-Ford algorithm has to loop $V - 1$ times to iterate through each edge and find the shortest path. Therefore, the outer loop has to iterate based on the number of vertices, while the inner loop has to iterate based on the number of edges, which results in a time complexity of $O(V \times E)$. Especially in the worst-case scenario where array updates occur in every iteration, this forces the algorithm to perform all $V - 1$ iteration without early termination.

3. Negative cycle detection

```
for (EdgeIndex e : edges) {
    if (dist[e.startIndex] == Double.POSITIVE_INFINITY) continue;
    if (dist[e.startIndex] + e.weight < dist[e.endIndex]) {
       return "Negative cycle found from vertex " + start + ". This path is invalid.";
    }
}
```

After the Bellman-Ford algorithm performs all edge relaxation operations, the algorithm then checks through all edges to ensure that there is no negative cycle formed. Since the `for` -loop iterates based on the number of edges, this results in a time complexity of $O(E)$.

4. Shortest path reconstruction

```
ArrayList<String> path = new ArrayList<>();
int current = endIdx;

while (current != -1) {
    path.addFirst(vertices.get(current).label());
    if (current == startIdx) break;
    current = prev[current];
}
```

Finally, the shortest path is constructed by backtracking the `prev` array. In the worst-case scenario, the length of the path will consist at most $V - 1$ number of vertices, resulting in a time complexity of $O(V)$ since the number of iterations of the `while` -loop depends on the number of vertices.

Overall, the combination of these process forms a total time complexity of:

$$T(V) = O(V) + O(E) + O(V \times E) + O(E) + O(V)$$

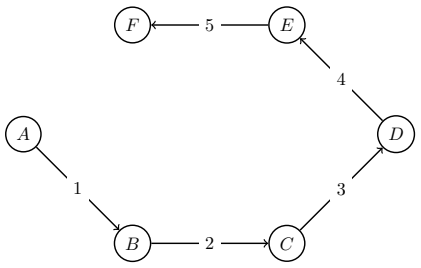
Since $O(V \times E)$ dominates the other terms, the overall worst-case time complexity is $\boldsymbol{O(VE)}$.

Best Case Analysis

As the code snippet in Figure 5 does not contain any mechanisms for early termination, the Bellman-Ford algorithm will always perform full edge relaxation processes, no matter the complexity of the input graph and the input sequence of edges. Hence, the time complexity of the algorithm remains $O(VE)$ for all scenarios. However, it is observed that the frequency of edge relaxation operations varies based on the graph complexity and edge input sequence. Hence, the best-case scenario is demonstrated using the graph in Figure 10 that allows the algorithm to find the shortest path with minimal distance updates.

Figure 10

Graph for Bellman-Ford's Best-Case Analysis Demonstration



Adjacency List Representation:

```
A -> [(A, B, 1)]
B -> [(B, C, 2)]
C -> [(C, D, 3)]
D -> [(D, E, 4)]
E -> [(E, F, 5)]
F -> []
```

The graph in Figure 10 represents the best-case scenario for the algorithm due to:

- **Topological order for edge input**
  With the edges being processed in an optimal order, all distances can be updated within the same iteration, thereby allowing the shortest path to be identified within one iteration.

- **No wasted updates**
  Each element in the distance array is updated only once, with no further changes taking place in subsequent iterations. If an early termination logic is applied to the Bellman-Ford algorithm, where the algorithm does not loop through edges that does not contribute to shorter distance, the time complexity of the algorithm can be further reduced to $O(E)$.

The trace table in Appendix C demonstrates the Bellman-Ford algorithm in best-case scenario, where the same process follows from initializing the `dist` and `prev` array, calculating the shortest path, checking for negative cycles and constructing the shortest path. Hence, the time complexity remains $\boldsymbol{O(VE)}$, with the distance updates in each iteration minimised.

Algorithm 2: Floyd-Warshall

The Java code snippet used to analyse the time and space complexities of the Floyd-Warshall algorithm is shown in Figure 11.

Figure 11

Code Snippet of Floyd-Warshall Algorithm for Time and Space Complexity Analysis

```java
public class Floyd_Warshall {
    public static String getShortestPath(double[][] distMatrix,
                                          ArrayList<Graph.Vertex> vertexList,
                                          Graph.Vertex start,
                                          Graph.Vertex end) {

        // Initialize next matrix
        int matrixSize = vertexList.size();
        Integer[][] nextMatrix = new Integer[matrixSize][matrixSize];
        for (int row = 0; row < matrixSize; row++) {
            for (int col = 0; col < matrixSize; col++) {
                if (row != col && distMatrix[row][col] < Double.POSITIVE_INFINITY) {
                    nextMatrix[row][col] = col;
                }
            }
        }

        // Floyd-Warshall algorithm to determine shortest path
        for (int middle = 0; middle < matrixSize; middle++) {
            for (int row = 0; row < matrixSize; row++) {
                for (int col = 0; col < matrixSize; col++) {
                    if (distMatrix[row][middle] + distMatrix[middle][col] < distMatrix[row][col]) {
                        distMatrix[row][col] = distMatrix[row][middle] + distMatrix[middle][col];
                        nextMatrix[row][col] = nextMatrix[row][middle];
                    }
                }
            }
        }

        // Check for invalid cases
        int startNodeIndex = vertexList.indexOf(start);
        int endNodeIndex = vertexList.indexOf(end);
        if (startNodeIndex == endNodeIndex)
            return "Distance from " + start + " to " + end + " is: 0";
        try {
            double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];
            if (shortestDistance == Double.POSITIVE_INFINITY) throw new ArrayIndexOutOfBoundsException();
        } catch (ArrayIndexOutOfBoundsException _) {
            return "Distance from " + start + " to " + end + " is: Not found";
        }
        double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];

        // Retrieve and return shortest path
        ArrayList<String> path = new ArrayList<>();
        path.add(start.label());
        int currentNodeIndex = startNodeIndex;
        while (currentNodeIndex != endNodeIndex) {
            Integer middleVertex = nextMatrix[currentNodeIndex][endNodeIndex];
            if (middleVertex == null)
                return "Error. No path is found.";
            path.add(vertexList.get(middleVertex).label());
            currentNodeIndex = middleVertex;
        }
        return "Distance from " + start + " to " + end + " is: " + shortestDistance +
                "\nThe shortest path is: " + String.join(" -> ", path);
    }
}
```

Algorithm Correctness

To verify the correctness of the Floyd-Warshall algorithm, two types of tests, namely invariant checking and empirical testing, are employed into the code implementation in Figure 11 to make sure that negative cycles and overflow issues do not affect the algorithm's correctness.

- **Invariant checking**

  To perform invariant checking on the Floyd-Warshall algorithm, a check must be made after each iteration of the outer loop so that errors can be captured early before the algorithm moves to the next intermediate node. Two checks, namely the negative cycle check to detect scenarios where a node can reach back to itself with a negative total cost and lead to inaccurate results, and the intermediate distance check to show correct updates towards the distance and next matrices, is demonstrated in Figure 12.

Figure 12

Invariant Checking for Floyd-Warshall Algorithm in Java

```
// Other parts of the function ...

// Floyd-Warshall algorithm begins here:
for (int middle = 0; middle < matrixSize; middle++) {
    for (int row = 0; row < matrixSize; row++) {
        for (int col = 0; col < matrixSize; col++) {
            if (distMatrix[row][middle] + distMatrix[middle][col] < distMatrix[row][col]) {
                distMatrix[row][col] = distMatrix[row][middle] + distMatrix[middle][col];
                nextMatrix[row][col] = nextMatrix[row][middle];
            }
        }
    }

    // ====== INVARIANT TESTING ======
    for (int row = 0; row < matrixSize; row++) {

        // Check 1: Negative cycle check - negative value in distance matrix diagonal => negative cycle exists
        assert distMatrix[row][row] >= 0 : "Invariant Violated: Negative cycle detected at node " + row;

        // Loop through the elements in each row
        for (int col = 0; col < matrixSize; col++) {

            // Check 2: Intermediate distance check
            // Use if-statement to make sure that the path through intermediary node exists
            if (distMatrix[row][middle] != Double.POSITIVE_INFINITY
                    && distMatrix[middle][col] != Double.POSITIVE_INFINITY) {

                // Perform checking to see if a shorter distance can be found
                assert distMatrix[row][col] <= distMatrix[row][middle] + distMatrix[middle][col] :
                        "Invariant Violated: Path through " + middle + " is shorter but not updated!";
            }
        }
    }

    // Output that checking passes
    System.out.println("Invariant checking [Iteration " + (middle + 1) + "]: PASS");
}

// This one is just to make the output look nicer (^_^)
System.out.println();

// Other parts of the function ...
```

Using the graph in Figure 13, invariant checking is performed on the Floyd-Warshall algorithm using the code snippet in Figure 12 and 14. Based on the output, the algorithm successfully passes all checks without running into any assertion errors, and hence, the algorithm passes the invariant checking with its correctness during execution proved.

Figure 13

An Example of Graph used to Demonstrate Invariant Checking

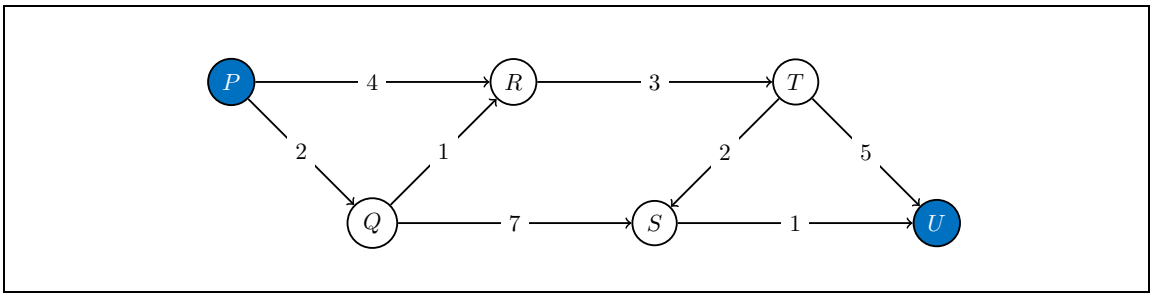


Figure 14

Code Snippet and Output of the Main Class for Invariant Checking Execution

```
public class Main {

    private static final ArrayList<GraphInList> adjacency_list_array = new ArrayList<>();
    private static final ArrayList<GraphInMatrix> adjacency_matrix_array = new ArrayList<>();

    public static void main(String[] args) {

        // A sample data for testing
        String example_input = """
        [(P, Q, 2); (P, R, 4); (Q, R, 1); (Q, S, 7); (R, T, 3); (S, U, 1); (T, S, 2); (T, U, 5)] | P | U
        """;

        // Convert the string data into graph and vertex objects
        preprocessing_steps(example_input);

        // Floyd-Warshall
        System.out.println("--- Floyd-Warshall: Graph ---");
        System.out.println(Floyd_Warshall.getShortestPath(
                adjacency_matrix_array.getFirst().matrix,
                adjacency_matrix_array.getFirst().vertexList,
                adjacency_matrix_array.getFirst().start,
                adjacency_matrix_array.getFirst().end
        ));
        System.out.println();
    }
}
```

```
--- Floyd-Warshall: Graph 1: ---
Invariant checking [Iteration 1]: PASS
Invariant checking [Iteration 2]: PASS
Invariant checking [Iteration 3]: PASS
Invariant checking [Iteration 4]: PASS
Invariant checking [Iteration 5]: PASS
Invariant checking [Iteration 6]: PASS

Distance from P to U is: 9.0
The shortest path is: P -> Q -> R -> T -> S -> U


Process finished with exit code 0
```

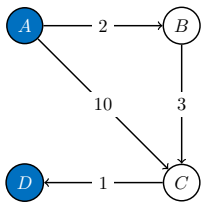
- **Empirical testing**

  To perform empirical testing, four graphs are prepared as shown in Figure 15, where each graph represents different scenarios that the algorithm may encounter during its execution. From Figure 16, it is observed that the algorithm's output tally with the expected outputs from the sample graphs, thereby supporting the algorithm's correctness when different types of graphs, such as disconnected graphs and graphs with negative edges, are inputted into the algorithm.

Figure 15

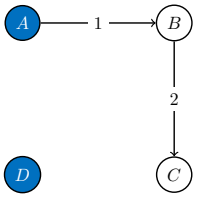
Sample Graphs Used for Empirical Testing

Graph 1: Normal Graph



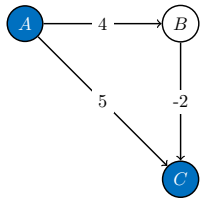

```
Start node                  : A
End node                    : D
Expected shortest distance  : 6
Expected shortest path      : A -> B -> C -> D
```

Graph 2: Disconnected Graph



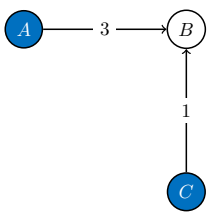

```
Start node                  : A
End node                    : D
Expected shortest distance  : -
Expected shortest path      : -
```

Graph 3: Graph with Negative Edges




```
Start node                  : A
End node                    : C
Expected shortest distance  : 2
Expected shortest path      : A -> B -> C
```

Graph 4: Graph with No Path




```
Start node                  : A
End node                    : C
Expected shortest distance  : -
Expected shortest path      : -
```

Figure 16

Code Snippet and Output of the Main Class for Empirical Testing Execution

```
public class Main {

    private static final ArrayList<GraphInList> adjacency_list_array = new ArrayList<>();
    private static final ArrayList<GraphInMatrix> adjacency_matrix_array = new ArrayList<>();

    public static void main(String[] args) {

        // A sample data for testing
        String example_input = """
        [(A, B, 2); (B, C, 3); (A, C, 10); (C, D, 1)] | A | D
        [(A, B, 1); (B, C, 2)] | A | D
        [(A, B, 4); (A, C, 5); (B, C, -2)] | A | C
        [(A, B, 3); (C, B, 1)] | A | C
        """;

        // Convert the string data into graph and vertex objects
        preprocessing_steps(example_input);

        // Floyd-Warshall
        for (int i = 0; i < adjacency_matrix_array.size(); i++) {
            System.out.println("--- Floyd-Warshall: Graph " + (i + 1) + " ---");
            System.out.println(Floyd_Warshall.getShortestPath(
                    adjacency_matrix_array.get(i).matrix,
                    adjacency_matrix_array.get(i).vertexList,
                    adjacency_matrix_array.get(i).start,
                    adjacency_matrix_array.get(i).end
            ));
            System.out.println();
        }
    }
}
```

```
--- Floyd-Warshall: Graph 1 ---
Invariant checking [Iteration 1]: PASS
Invariant checking [Iteration 2]: PASS
Invariant checking [Iteration 3]: PASS
Invariant checking [Iteration 4]: PASS

Distance from A to D is: 6.0
The shortest path is: A -> B -> C -> D

--- Floyd-Warshall: Graph 2 ---
Invariant checking [Iteration 1]: PASS
Invariant checking [Iteration 2]: PASS
Invariant checking [Iteration 3]: PASS

Distance from A to D is: Not found

--- Floyd-Warshall: Graph 3 ---
Invariant checking [Iteration 1]: PASS
Invariant checking [Iteration 2]: PASS
Invariant checking [Iteration 3]: PASS

Distance from A to C is: 2.0
The shortest path is: A -> B -> C

--- Floyd-Warshall: Graph 4 ---
Invariant checking [Iteration 1]: PASS
Invariant checking [Iteration 2]: PASS
Invariant checking [Iteration 3]: PASS

Distance from A to C is: Not found

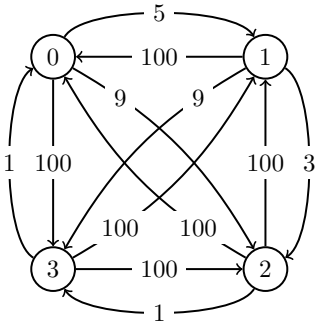

Process finished with exit code 0
```

Worst Case Analysis: Step-by-Step Trace

To determine the shortest distance between two points, the Floyd-Warshall algorithm computes the shortest distance between all the possible combinations of node pairs and updates the distance matrix if a shorter path is identified, then lastly indexes the distance matrix to return the shortest distance between two specific points (Joshi, 2026). As the algorithm must perform checking on each possible node pair with intermediary node before retrieving the shortest distance, the Floyd-Warshall algorithm is unable to terminate early no matter the complexity of input graphs, causing its time complexity to remain similar across different scenarios. However, since the number of updates towards the distance matrix varies across different graph structure, a graph with suboptimal direct edge weights, as shown in Figure 17, is constructed to demonstrate maximal matrix update operations within the algorithm.

Figure 17

Graph for Floyd-Warshall's Worst-Case Analysis Demonstration



Adjacency Matrix Representation:

$$\text{Distance matrix} = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array} \begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 100 & 100 & 0 \end{bmatrix} \end{array}$$

The direct edges of the graph in Figure 17 are suboptimal due to the following scenarios:

- **Diagonal traversal**

  When traversing diagonally, the direct edges (e.g., from 2 to 0) have a cost of 100 units. However, by traversing through the outer ring to reach the diagonal node, the cost reduces drastically to 2 units, making the direct edges of diagonal nodes suboptimal.

- **Reversed traversal**

  When traversing in a reversed manner, the direct edges (e.g., from 1 to 0) have a cost of 100 units. However, traversing in an opposite direction costs 5 units to reach the destination node, making the direct edges to be suboptimal too.

Due to the high number of suboptimal direct edges in the graph, the Floyd-Warshall algorithm has to undergo frequent edge relaxation operations in an iterative manner to calculate the shortest distance between each possible node pairs. From the trace table in Appendix D that demonstrates the Floyd-Warshall algorithm in action step-by-step, it is observed that:

- **A high number of elements in the distance matrix are updated**
  The distance matrix has gone through a significant change throughout the process:

$$\begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array}\!\!\begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 100 & 100 & 0 \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \end{array}\!\!\begin{array}{c} \begin{array}{cccc} 0 & 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 2 & 7 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix} \end{array}$$

  By comparing the initial and the resultant distance matrix, it is observed that half of the elements (8 out of 16) in the distance matrix are replaced by a lower weight. The high number of element updates indicates that the algorithm has undergone a high number of edge relaxation operations to identify the shortest paths for the corresponding nodes, thereby demonstrating an increased number of distance updates within the algorithm.

- **Certain entries are updated multiple times before converging to their final values**
  It is also observed that, several elements in the distance matrix have undergone multiple updates throughout the execution of the algorithm, such as:

$$\begin{array}{c} \\ 0 \\ \vdots \end{array}\!\!\begin{array}{c} \begin{array}{cc} \cdots & 3 \end{array} \\ \begin{bmatrix} \cdots & 100 \\ \iddots & \vdots \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 0 \\ \vdots \end{array}\!\!\begin{array}{c} \begin{array}{cc} \cdots & 3 \end{array} \\ \begin{bmatrix} \cdots & 14 \\ \iddots & \vdots \end{bmatrix} \end{array} \longrightarrow \begin{array}{c} \\ 0 \\ \vdots \end{array}\!\!\begin{array}{c} \begin{array}{cc} \cdots & 3 \end{array} \\ \begin{bmatrix} \cdots & 9 \\ \iddots & \vdots \end{bmatrix} \end{array}$$

  These repetitive updates towards the same element indicates that the algorithm is incrementally improving the weights as it iterates through different intermediate nodes. However, since the algorithm has to perform multiple update operations, this contributes to an increased number of distance updates simultaneously.

The frequent and multiple updates towards the distance matrix throughout the iterations of the Floyd-Warshall algorithm indicates the high number of edge relaxation operations performed, allowing the graph in Figure 17 to demonstrate the worst-case behaviour in terms of matrix update operations.

Worst Case Analysis: Time Complexity

Based on the code snippet in Figure 11, four steps are involved in the Floyd-Warshall algorithm to determine the shortest path and distance between two nodes:

1. Next matrix initialization

```
int matrixSize = vertexList.size();
Integer[][] nextMatrix = new Integer[matrixSize][matrixSize];
for (int row = 0; row < matrixSize; row++) {
    for (int col = 0; col < matrixSize; col++) {
        if (row != col && distMatrix[row][col] < Double.POSITIVE_INFINITY) {
            nextMatrix[row][col] = col;
        }
    }
}
```

Before executing the algorithm, the code first identifies the number of vertices in the graph, represented using the symbol V, to be used for subsequent operations. Determining the matrix size takes $O(1)$ time, and the code contains double nested loops that iterates over V, resulting in a complexity of $O(V^2)$. Hence, the combination of both operations yield a time complexity of:

$$O(1) + O(V^2) = O(V^2)$$

2. Execution of Floyd-Warshall algorithm

```
for (int middle = 0; middle < matrixSize; middle++) {
    for (int row = 0; row < matrixSize; row++) {
        for (int col = 0; col < matrixSize; col++) {
            if (distMatrix[row][middle] + distMatrix[middle][col] < distMatrix[row][col]) {
                distMatrix[row][col] = distMatrix[row][middle] + distMatrix[middle][col];
                nextMatrix[row][col] = nextMatrix[row][middle];
            }
        }
    }
}
```

The Floyd-Warshall algorithm calculates the shortest path by considering all the possible combinations of intermediate nodes, represented by V, for every node pair ($V \times V$). This forms a triple nested loop, where each loop iterates V times, contributing to a time complexity of:

$$O(V \times V \times V) = O(V^3)$$

3. Checking for special cases

```
int startNodeIndex = vertexList.indexOf(start);
int endNodeIndex = vertexList.indexOf(end);
if (startNodeIndex == endNodeIndex)
    return "Distance from " + start + " to " + end + " is: 0";
try {
    double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];
    if (shortestDistance == Double.POSITIVE_INFINITY) throw new ArrayIndexOutOfBoundsException();
} catch (ArrayIndexOutOfBoundsException _) {
    return "Distance from " + start + " to " + end + " is: Not found";
}
double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];
```

Once the distance and next matrices are constructed, the algorithm then retrieves the index of the start node and end node by performing a linear search towards the `vertexList` array, where each operation takes $O(V)$ time for the worst-case scenario. The comparisons in the two `if` -statements, and the code to index an element from the distance matrix, takes $O(1)$ time each. The combination of these operations yields:

$$O(V) + O(V) + O(1) + O(1) + O(1) + O(1) = O(V)$$

4. Shortest path retrieval

```
ArrayList<String> path = new ArrayList<>();
path.add(start.label());
int currentNodeIndex = startNodeIndex;
while (currentNodeIndex != endNodeIndex) {
    Integer middleVertex = nextMatrix[currentNodeIndex][endNodeIndex];
    if (middleVertex == null)
        return "Error. No path is found.";
    path.add(vertexList.get(middleVertex).label());
    currentNodeIndex = middleVertex;
}
return "Distance from " + start + " to " + end + " is: " + shortestDistance +
        "\nThe shortest path is: " + String.join(" -> ", path);
```

Lastly, to find the path with the shortest distance between two nodes, the algorithm performs backtracking on the next matrix. Adding the start node to the path array takes a complexity of $O(1)$, and the `while` -loop takes $O(V)$ to run in the worst-case scenario, when all nodes act as intermediate nodes and contribute to the shortest path, which gives:

$$O(1) + O(V) = O(V)$$

In overall, combining all the time complexity functions yields:

$$T(V) = O\left(V^2\right) + O\left(V^3\right) + O(V) + O(V)$$

and since $O(V^3)$ dominates the other terms, the overall worst-case time complexity is $\boldsymbol{O\left(V^3\right)}$.

Best Case Analysis

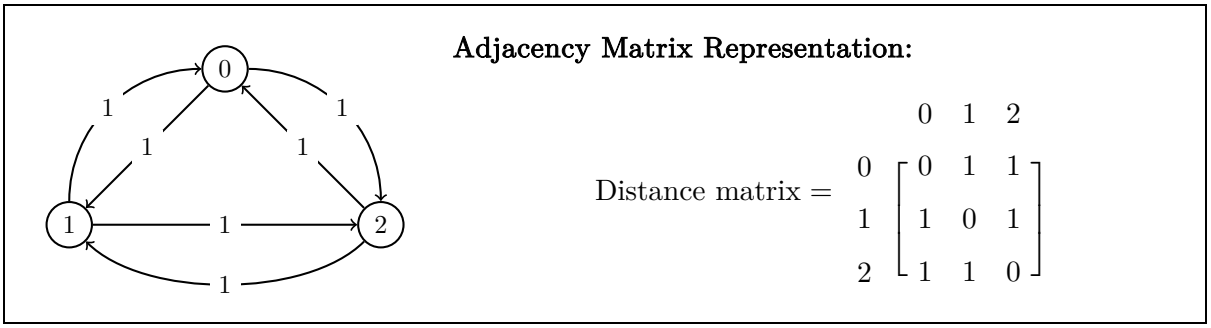
As the Floyd-Warshall algorithm will always perform the triple-nested loops to go through all the possible combinations of node pairs with each intermediate node, the time complexity of the algorithm remains:

$$O(V^3)$$

for all cases, regardless of the graph inputted to the algorithm. However, since the frequency of matrix update activities varies depending on graph complexity, the demonstration of the best-case scenario focuses on minimizing matrix update operations involved in the algorithm, which is demonstrated using the graph in Figure 18.

Figure 18

Graph with Uniform Weights for Best-Case Analysis Demonstration



The graph in Figure 18 is complete and has a uniform weight of 1 across all edges. This indicates that all the direct edges between each node pair represent the shortest path, and the Floyd-Warshall algorithm will not have to update the distance matrix. Such a behaviour can be proved by looking at the trace table in Appendix E, which exhibits the following:

- **Evaluation for edge relaxation condition is always false**
  When comparing the path distance with and without intermediary node, the algorithm always gives `false`, indicating that intermediary node never yields a shorter distance.

- **High number of iterations**
  The algorithm evaluates the checking process for exactly 27 times, tallying with V^3.

These operations in the trace table in Appendix E show that the graph demonstrates the best-case scenario for the algorithm, where the algorithm exhibits the time complexity of $\boldsymbol{O(V^3)}$ while having minimized edge relaxation operations that results in reduced matrix updates.

Part C: Comparison and Evaluation between Algorithms

Performance Comparison

Based on the findings from Part B, the time complexity for both Bellman-Ford and Floyd-Warshall algorithms are summarized in Table 1.

Table 1

Time Complexity for Bellman-Ford and Floyd-Warshall Algorithms

Algorithm	Best-Case Time Complexity	Worst-Case Time Complexity
Bellman-Ford	$O(VE)$	$O(VE)$
Floyd-Warshall	$O(V^3)$	$O(V^3)$

By comparing the best-case and worst-case time complexities, it is observed that:

- **Both algorithms have a consistent asymptotic performance**
  As the best-case and worst-case time complexity of both algorithm remains the same, this indicates that the performance of both algorithm is only dependent on the size of the graph but not its variations. This is a crucial characteristic that ensures the predictability and reliability of algorithms when applied in real-world scenarios, where systems will not encounter significant fluctuations in their performance when handling different data.

- **Bellman-Ford algorithm is efficient when number of edges is small**
  The $O(VE)$ time complexity of Bellman-Ford algorithm indicates that the algorithm's performance increases proportionally when the number of vertices and edges increases. Especially in the case where the number of edges, E is small, the time complexity of Bellman-Ford algorithm grows slower than Floyd-Warshall, making the algorithm to be more effective, not only due to its shorter execution time, but also due to the lesser memory used as Bellman-Ford uses arrays to perform operations and store outputs.

- **Performance of Floyd-Warshall algorithm does not depend on the number of edges**
  As the time complexity of the Floyd-Warshall algorithm only involves V, the algorithm will yield the same processing time regardless of the number of edges in the graph. This makes the algorithm more suitable when a graph has a high number of edges $(E \approx V^2)$ that makes the performance of Bellman-Ford to be similar to Floyd-Warshall, as well as in cases where the shortest distances between all node pairs are required to be calculated.

To demonstrate the time complexity performance of both algorithms, the code snippet was slightly modified to record the algorithm execution time, as shown in Appendix F. Based on the output, it is observed that:

Algorithm	Average Execution Time (ns)
Bellman-Ford	668.10075
Floyd-Warshall	830.192

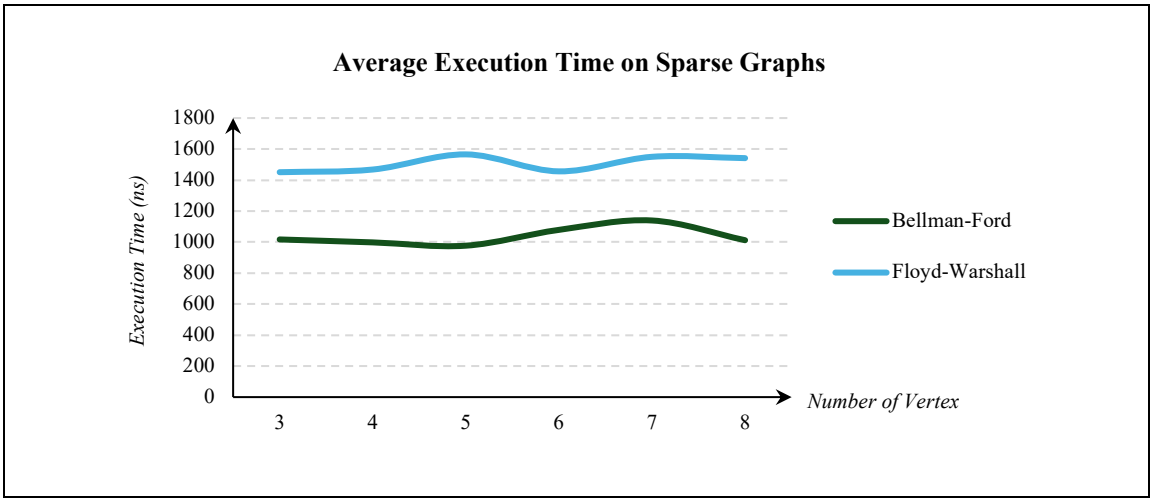
On average, the Bellman-Ford algorithm has a lower execution time compared to the Floyd-Warshall algorithm, as the samples used generally have a low number of edges, where the number of edges, E is relatively smaller than the square of the number of vertices, V^2. This makes the Bellman-Ford algorithm, with a time complexity of $O(VE)$, to not perform as many computations as the Floyd-Warshall algorithm with a time complexity of $O(V^3)$. Therefore, to dive deeper into the difference in performance between the two algorithms, the analysis is divided into two categories:

- **Sparse graph**

  The sample graphs used in this scenario has a low number of edges, where the number of edges is approximately linear to the number of vertices ($E \approx V$) that makes the Bellman-Ford algorithm to experience lesser computational overhead if compared to Floyd-Warshall. This finding is also aligned with the performance graph of both algorithms in Figure 19.

Figure 19

Average Execution Time of Bellman-Ford and Floyd-Warshall on Sparse Graphs

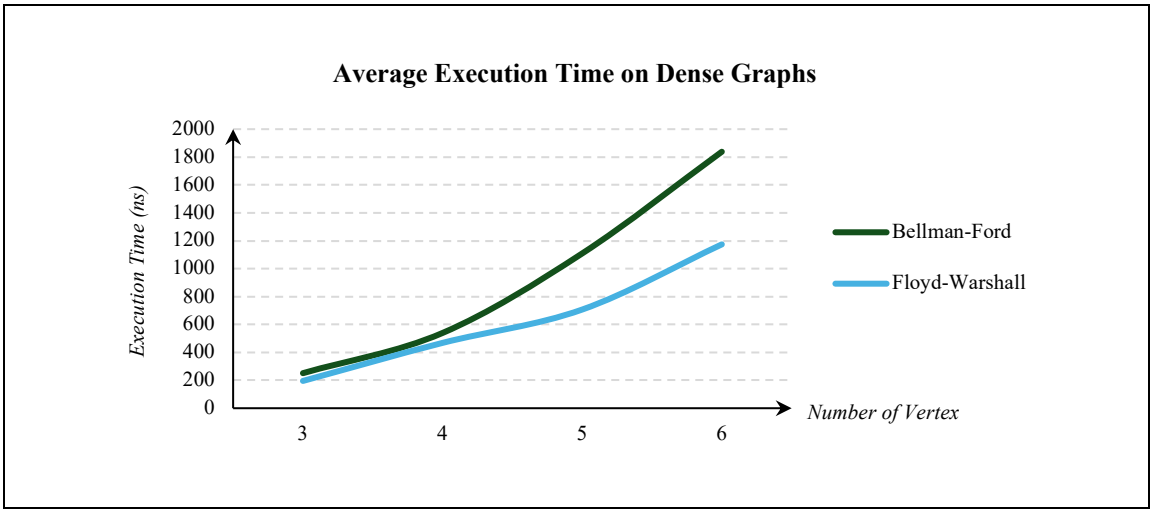


- **Dense graph**

  However, on dense graphs, when sample graphs have number of edges approximately to V^2, this makes the time complexity of the Bellman-Ford algorithm, $O(VE)$, to be close to $O(V^3)$, making the performance of Bellman-Ford to be similar to Floyd-Warshall. This is aligned with the performance graph in Figure 20, where interestingly, when the number of vertices increases, the Floyd-Warshall algorithm becomes more effective compared to Bellman-Ford as it performs structured computational activities by constantly iterating and updating the fixed-size matrix, thereby improving cache locality to yield better results.

Figure 20

Average Execution Time of Bellman-Ford and Floyd-Warshall on Dense Graphs



Due to the similar time complexity of both algorithms, when dense graphs are involved in the calculation of the shortest path, the choice of algorithm lies on the specific requirements of the problem, where:

- Bellman-Ford is often preferred in determining the single-source shortest path, as the algorithm does not perform computation on all vertex pairs. Moreover, since the algorithm requires only two arrays, namely the distance array and the predecessor array, the algorithm has a space complexity of $O(V)$ that is more memory efficient.

- Floyd-Warshall is often preferred when the shortest path between all node pairs is required due to the nature of the algorithm that has the ability to directly calculate the shortest distance between any node combination, unlike the Bellman-Ford algorithm that performs its edge relaxation operations by relying on the starting node.

Algorithm Use Case

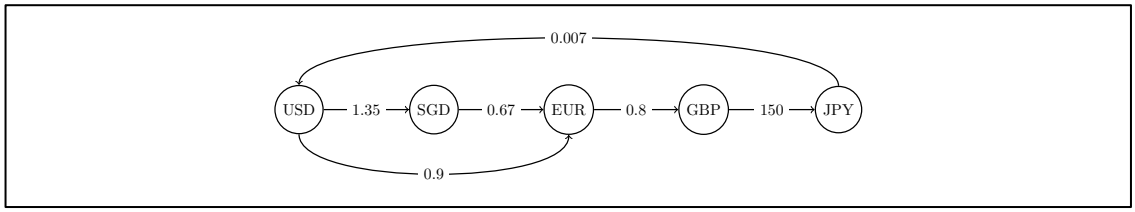
Based on the performance evaluation, each algorithm has its own best use case:

- **Bellman-Ford algorithm**

  As the Bellman-Ford algorithm is able to handle sparse graph and negative cycles more effectively than Floyd-Warshall, the Bellman-Ford algorithm can be applied in a currency exchange system. Taking the example in Figure 21, the negative cycle detection in Bellman-Ford algorithm allows the system to detect currency arbitrage opportunities, while the shortest path detection allows the system to calculate the most profitable route when exchanging between different currencies (Rusli, 2025).

Figure 21

An Example of Exchange Rates between Different Currencies in Graph Representation



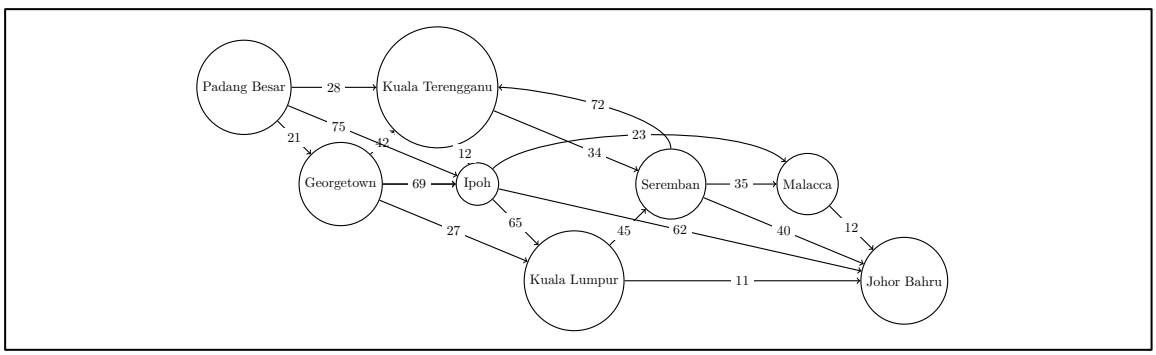
Note. Exchange rates can be negative after log-transformation.

- **Floyd-Warshall algorithm**

  The Floyd-Warshall algorithm, on the other hand, is better in handling dense graphs with complex connections but without negative cycles. Hence, the algorithm is suitable to be applied in multi-source graphs, such as the transportation network shown in Figure 22, to effectively compute the shortest paths between all pairs of cities within a consistent time.

Figure 22

An Example of Transportation Network in Graph Representation



Improvement Suggestions

Some of the improvements suggested for each algorithm include:

- **Early termination for Bellman-Ford algorithm**

  The naïve Bellman-Ford algorithm must execute full edge relaxation operations to retrieve the shortest path to the destination, where if the shortest path is found in early iterations, the subsequent loops do not add any value. Hence, by adding a flag to check the update status of the current iteration, the early termination allows the algorithm's best-case time complexity to be improved to $O(E)$ since the algorithm will only have to iterate through each edge once, meanwhile maintaining the same space complexity of $O(V)$.

```
// Bellman-Ford algorithm
for (int i = 0; i < verticesCount - 1; i++) {

    boolean updated = false; // Flag-check to stop early

    for (EdgeIndex edge : edges) {
       if (dist[edge.startIndex] == Double.POSITIVE_INFINITY) continue;
       double newDist = dist[edge.startIndex] + edge.weight;
       if (newDist < dist[edge.endIndex]) {
          dist[edge.endIndex] = newDist;
          prev[edge.endIndex] = edge.startIndex;

          updated = true; // Iteration continues when changes take place
       }
    }

    if (!updated) break; // If no updates, shortest paths are found, can exit early
}
```

- **Negative cycle handling for Floyd-Warshall algorithm**

  Although Floyd-Warshall can handle dense graphs efficiently, it yields incorrect results when negative cycle exists. Therefore, a mechanism to detect negative cycles can be implemented by checking if the diagonal of the distance matrix has negative values. Although this increases the time complexity by $O(V)$, the overall time complexity and space complexity remains unaffected with the additional checking that ensures correctness.

```
// Floyd-Warshall algorithm runs here...

// An optimization to check for negative cycles on diagonal after algorithm
for (int i = 0; i < matrixSize; i++) {
    if (distMatrix[i][i] < 0) {
        return "Negative cycle detected. Shortest path is invalid.";
    }
}

// Shortest path retrieval continues here...
```

Conclusion

In summary, this report looks into how the Bellman-Ford and Floyd-Warshall algorithm calculates the shortest distance and path between two nodes in simple directed weighted graphs, where the Bellman-Ford algorithm has a time complexity of $O(VE)$ in best and worst-case scenarios, while the Floyd-Warshall algorithm has a time complexity of $O(V^3)$ in best and worst-case scenarios. The performance of both algorithms is compared and evaluated with recommendations on their best use cases and further improvements, so that both algorithms can be appropriately applied in real-world system applications and bring convenience to users. It is hoped that the insights outlined in this report will be able to enhance readers' understanding on different types of shortest path algorithms and support their choice when selecting for the appropriate algorithm to be implemented in different scenarios.

References

Bharathi, D. S., & Kaviya, A. (2025, October). Application of Graph Theory in Real Life. *Wesleyan Journal of Research, 14*(1), 41-46.

Cyfrin. (n.d.). *Invariant Test*. Retrieved from Cyfrin: https://www.cyfrin.io/glossary/invariant-test

Hyperskill. (n.d.). *Weighted graph*. Retrieved from Hyperskill: https://hyperskill.org/learn/step/5645

Joshi, S. (2026, January 24). *Floyd–Warshall Algorithm: Understanding All-Pairs Shortest Paths*. Retrieved from Medium: https://medium.com/@shrutitech98/floyd-warshall-algorithm-understanding-all-pairs-shortest-paths-13f7060c00e7

Jukna, S., & Schnitger, G. (2016, May 16). On the optimality of Bellman–Ford–Moore shortest path algorithm. *Theoretical Computer Science, 628*, 101-109.

Maung, H. (2025, July 28). *What is a Graph?* Retrieved from Medium: https://medium.com/@hsuzarnimaung/what-is-a-graph-a2d998a6b7e4

Meegle. (2026, February 9). *Algorithm Testing Methods*. Retrieved from Meegle: https://www.meegle.com/en_us/topics/algorithm/algorithm-testing-methods

Nayyar, R. (2024, December 17). *Representing Graphs in Data Structures*. Retrieved from Great Learning: https://www.mygreatlearning.com/blog/representing-graphs-in-data-structures/

NxtWave. (2025, November 20). *Application of Graph in Data Structures: Components & Types*. Retrieved from NxtWave: https://www.ccbp.in/blog/articles/application-of-graph-in-data-structure

Rusli, N. J. (2025, January 5). Arbitrage Detection in Financial Markets: A Graph Theory Approach Using the Bellman-Ford Algorithm. *Makalah IF1220 Matematika Diskrit*.

Appendices

Appendix A

Code Implementation of Graph in Java

```
public class Graph {

    // Adjacency list to represent graph
    private final Map<Vertex, ArrayList<Edge>> adjacencyList = new HashMap<>();

    // Constructor for graph
    public Graph(String inputText) {
        inputText = inputText.trim().substring(1, inputText.length() - 1);
        String[] splitArray = inputText.split("; ");

        ArrayList<Edge> edgeArray = new ArrayList<>();
        for (String textEdge: splitArray) {
            Edge edge = Edge.fromText(textEdge);
            edgeArray.add(edge);
        }

        for (Edge edge: edgeArray) {
            adjacencyList.putIfAbsent(edge.startNode, new ArrayList<>());
            adjacencyList.putIfAbsent(edge.endNode, new ArrayList<>());
            adjacencyList.get(edge.startNode).add(edge);
        }
    }

    // A function to print the graph details
    public void printGraph() {
        adjacencyList.forEach((vertex, edgeArray) -> {
            System.out.print(vertex.label() + " -> [");
            for (Edge edge : edgeArray) {
                System.out.print("(" + edge.endNode.label() + ", w=" + edge.weight + "), ");
            }
            System.out.print("]\n");
        });
    }

    // A function to convert graphs from adjacency list to matrix
    public double[][] convertToMatrix() {

        int vertexCount = adjacencyList.size();
        double[][] matrix = new double[vertexCount][vertexCount];
        for (double[] row: matrix) {
            Arrays.fill(row, Double.POSITIVE_INFINITY);
        }
        for (int i = 0; i < vertexCount; i++) {
            matrix[i][i] = 0;
        }

        ArrayList<Vertex> sortedVertexList = this.getSortedVertices();
        Map<Vertex, Integer> vertexWithIndex = new HashMap<>();
        for (int i = 0; i < vertexCount; i++) {
            vertexWithIndex.put(sortedVertexList.get(i), i);
        }

        for (Map.Entry<Vertex, ArrayList<Edge>> entry : adjacencyList.entrySet()) {
            for (Edge edge: entry.getValue()) {
                int startingNodeIndex = vertexWithIndex.get(edge.startNode);
                int endingNodeIndex = vertexWithIndex.get(edge.endNode);
                matrix[startingNodeIndex][endingNodeIndex] = edge.weight;
            }
        }
        return matrix;
    }


    // A method to sort the list of vertices
    public ArrayList<Vertex> getSortedVertices() {
        ArrayList<Vertex> vertexList = new ArrayList<>(adjacencyList.keySet());
        vertexList.sort((a, b) -> a.label.compareToIgnoreCase(b.label));
        return vertexList;
    }
```

```
    // A function to retrieve the graph edges
    public ArrayList<Edge> getEdges() {
        ArrayList<Edge> edges = new ArrayList<>();
        adjacencyList.forEach((vertex, edgeArray) -> {
            for (Edge edge : edgeArray) {
                edges.add(new Edge(edge.startNode, edge.endNode, edge.weight));
            }
        });
        return edges;
    }

    // Class (known as record) for vertex
    public record Vertex(String label) {

        private static final ArrayList<Vertex> vertexArray = new ArrayList<>();

        public Vertex(String label) {
            this.label = label.trim();
            vertexArray.add(this);
        }

        public static Vertex find(String input) {
            input = input.strip().toLowerCase();
            for (Vertex vertex : vertexArray) {
                if (vertex.label.toLowerCase().equals(input)) return vertex;
            }
            return null;
        }

        public String toString() {
            return label;
        }
    }

    // Class for edge
    public static class Edge {
        Vertex startNode;
        Vertex endNode;
        double weight;

        public Edge(Vertex startNode, Vertex endNode, double weight) {
            this.startNode = startNode;
            this.endNode = endNode;
            this.weight = weight;
        }

        public static Edge fromText(String input) {
            input = input.substring(1, input.length() - 1);
            String[] splitElements = input.split(",");
            Vertex startVertex = switch (Vertex.find(splitElements[0])) {
                case Vertex found -> found;
                case null -> new Vertex(splitElements[0].trim());
            };
            Vertex endVertex = switch (Vertex.find(splitElements[1])) {
                case Vertex found -> found;
                case null -> new Vertex(splitElements[1].trim());
            };
            double weight = Double.parseDouble(splitElements[2].trim());
            return new Edge(startVertex, endVertex, weight);
        }
    }
}
```

Appendix B

Trace Table for Worst Case Scenario of Bellman-Ford Algorithm

The code implementation is shown in Figure 5, and the graph used for this trace table is shown in Figure 9.

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	Status	Output
Initiation	-	0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1		
Iteration 1	(C, D, 1)	0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1	Skipped	
	(E, F, 1)	0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1	Skipped	
	(D, E, 1)	0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1	Skipped	
	(B, C, 1)	0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1	Skipped	
	(A, B, 1)	0	-1	1	0	∞	-1	∞	-1	∞	-1	∞	-1	Updated	
	(A, C, 100)	0	-1	1	0	100	0	∞	-1	∞	-1	∞	-1	Updated	
	(A, D, 100)	0	-1	1	0	100	0	100	0	∞	-1	∞	-1	Updated	
	(A, E, 100)	0	-1	1	0	100	0	100	0	100	0	∞	-1	Updated	
	(A, F, 100)	0	-1	1	0	100	0	100	0	100	0	100	0	Updated	
															dist = [0, 1, 100, 100, 100, 100] prev = [-1, 0, 0, 0, 0, 0]
Iteration 2	(C, D, 1)	0	-1	1	0	100	0	100	0	100	0	100	0	Rejected	
	(E, F, 1)	0	-1	1	0	100	0	100	0	100	0	100	0	Rejected	
	(D, E, 1)	0	-1	1	0	100	0	100	0	100	0	100	0	Rejected	
	(B, C, 1)	0	-1	1	0	2	1	100	0	100	0	100	0	Updated	

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	Status	Output
Iteration 2	(A, B, 1)	0	-1	1	0	2	1	100	0	100	0	100	0	Skipped	
	(A, C, 100)	0	-1	1	0	2	1	100	0	100	0	100	0	Skipped	
	(A, D, 100)	0	-1	1	0	2	1	100	0	100	0	100	0	Skipped	
	(A, E, 100)	0	-1	1	0	2	1	100	0	100	0	100	0	Skipped	
	(A, F, 100)	0	-1	1	0	2	1	100	0	100	0	100	0	Skipped	
															dist = [0, 1, 2, 100, 100, 100] prev = [-1, 0, 1, 0, 0, 0]
Iteration 3	(C, D, 1)	0	-1	1	0	2	1	3	2	100	0	100	0	Updated	
	(E, F, 1)	0	-1	1	0	2	1	3	2	100	0	100	0	Rejected	
	(D, E, 1)	0	-1	1	0	2	1	3	2	4	3	100	0	Updated	
	(B, C, 1)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(A, B, 1)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(A, C, 100)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(A, D, 100)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(A, E, 100)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(A, F, 100)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
															dist = [0, 1, 2, 3, 4, 100] prev = [-1, 0, 1, 2, 3, 0]
Iteration 4	(C, D, 1)	0	-1	1	0	2	1	3	2	4	3	100	0	Skipped	
	(E, F, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Updated	

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	Status	Output
Iteration 4	(D, E, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(B, C, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, B, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, C, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, D, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, E, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, F, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
															dist = [0, 1, 2, 3, 4, 5] prev = [-1, 0, 1, 2, 3, 4]
Iteration 5	(C, D, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(E, F, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(D, E, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(B, C, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, B, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, C, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, D, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, E, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	
	(A, F, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Skipped	

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	Status	Output
Iteration 5														All shortest distance found.	dist = [0, 1, 2, 3, 4, 5] prev = [-1, 0, 1, 2, 3, 4]
Validation	(C, D, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~2 + 1 < 3~~
	(E, F, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~4 + 1 < 5~~
	(D, E, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~3 + 1 < 4~~
	(B, C, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~1 + 1 < 2~~
	(A, B, 1)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~0 + 1 < 0~~
	(A, C, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~0 + 100 < 2~~
	(A, D, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~0 + 100 < 3~~
	(A, E, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~0 + 100 < 4~~
	(A, F, 100)	0	-1	1	0	2	1	3	2	4	3	5	4	Passed	~~0 + 100 < 5~~
Shortest path retrieval															**Distance from A to F is: 5** **The shortest path is: A -> B -> C -> D -> E -> F**

Appendix C

Trace Table for Best Case Scenario of Bellman-Ford Algorithm

The code implementation is shown in Figure 5, and the graph used for this trace table is shown in Figure 10.

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	status	output
Initiation		0	-1	∞	-1	∞	-1	∞	-1	∞	-1	∞	-1		
Iteration 1	(A, B, 1)	0	-1	1	0	∞	-1	∞	-1	∞	-1	∞	-1	Updated	
	(B, C, 2)	0	-1	1	0	3	1	∞	-1	∞	-1	∞	-1	Updated	
	(C, D, 3)	0	-1	1	0	3	1	6	2	∞	-1	∞	-1	Updated	
	(D, E, 4)	0	-1	1	0	3	1	6	2	10	3	∞	-1	Updated	
	(E, F, 5)	0	-1	1	0	3	1	6	2	10	3	15	4	Updated	
															dist = [0, 1, 3, 6, 10, 15] prev = [-1, 0, 1, 2, 3, 4]
Iteration 2, 3, 4	(A, B, 1)	0	-1	1	0	3	1	6	2	10	3	15	4	Skipped	
	(B, C, 2)	0	-1	1	0	3	1	6	2	10	3	15	4	Skipped	
	(C, D, 3)	0	-1	1	0	3	1	6	2	10	3	15	4	Skipped	
	(D, E, 4)	0	-1	1	0	3	1	6	2	10	3	15	4	Skipped	
	(E, F, 5)	0	-1	1	0	3	1	6	2	10	3	15	4	Skipped	
														All shortest distance found.	dist = [0, 1, 3, 6, 10, 15] prev = [-1, 0, 1, 2, 3, 4]

Stage	Edge	dist[0]	prev[0]	dist[1]	prev[1]	dist[2]	prev[2]	dist[3]	prev[3]	dist[4]	prev[4]	dist[5]	prev[5]	status	output
Validation	(A, B, 1)	0	-1	1	0	3	1	6	2	10	3	15	4	Passed	~~0 + 1 < 1~~
	(B, C, 2)	0	-1	1	0	3	1	6	2	10	3	15	4	Passed	~~1 + 2 < 3~~
	(C, D, 3)	0	-1	1	0	3	1	6	2	10	3	15	4	Passed	~~3 + 3 < 6~~
	(D, E, 4)	0	-1	1	0	3	1	6	2	10	3	15	4	Passed	~~6 + 4 < 10~~
	(E, F, 5)	0	-1	1	0	3	1	6	2	10	3	15	4	Passed	~~10 + 5 < 15~~
Shortest path retrieval															**Distance from A to F is: 15** **The shortest path is: A -> B -> C -> D -> E -> F**

Appendix D

Trace Table for Worst Case Scenario of Floyd-Warshall Algorithm

The trace table below uses `i`, `j`, and `k` to represent the row index, column index, and intermediary node respectively, which correspond to the variables `row`, `col`, and `middle` in code (see Figure 11 for code implementation and Figure 17 for the graph used for trace table).

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Data initiation	-	$\begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 100 & 100 & 0 \end{bmatrix}$									3	1	
Next matrix initialization	7 - 16	$\begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 100 & 100 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 2 & 3 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 1 & 2 & - \end{bmatrix}$										
Node 0 as intermediary	19 – 21			0	0	0							
	22						0	0	0	False			
	21			0	0	1							
	22						0	5	5	False			
	21			0	0	2							
	22						0	9	9	False			
	21			0	0	3							
	22						0	100	100	False			
	20 – 21			1	0	0							
	22						100	0	100	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 0 as intermediary	21			1	0	1							
	22						100	5	0	False			
	21			1	0	2							
	22						100	9	3	False			
	21			1	0	3							
	22						100	100	9	False			
	20 – 21			2	0	0							
	22						100	0	100	False			
	21			2	0	1							
	22						100	5	100	False			
	21			2	0	2							
	22						100	9	0	False			
	21			2	0	3							
	22						100	100	1	False			
	20 – 21			3	0	0							
	22						1	0	1	False			
	21			3	0	1							
	22						1	5	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 100 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 2 & 3 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 2 & - \end{bmatrix}$										

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 0 as intermediary	21			3	0	2							
	22						1	9	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 9 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 10 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 2 & 3 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			3	0	3							
	22						1	100	0	False			
Node 1 as intermediary	19 – 21			0	1	0							
	22						5	100	0	False			
	21			0	1	1							
	22						5	0	5	False			
	21			0	1	2							
	22						5	3	9	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 100 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 10 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 3 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			0	1	3							
	22						5	9	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 14 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 10 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 1 as intermediary	20 – 21			1	1	0							
	22						0	100	100	False			
	21			1	1	1							
	22						0	0	0	False			
	21			1	1	2							
	22						0	3	3	False			
	21			1	1	3							
	22						0	9	9	False			
	20 – 21			2	1	0							
	22						100	100	100	False			
	21			2	1	1							
	22						100	0	100	False			
	21			2	1	2							
	22						100	3	0	False			
	21			2	1	3							
	22						100	9	1	False			
	20 – 21			3	1	0							
	22						6	100	1	False			
	21			3	1	1							
	22						6	0	6	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 1 as intermediary	21			3	1	2							
	22						6	3	10	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 14 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			3	1	3							
	22						6	9	0	False			
Node 2 as intermediary	19 – 21			0	2	0							
	22						8	100	0	False			
	21			0	2	1							
	22						8	100	5	False			
	21			0	2	2							
	22						8	0	8	False			
	21			0	2	3							
	22						8	1	14	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 100 & 0 & 3 & 9 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 0 & - & 2 & 3 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	20 – 21			1	2	0							
	22						3	100	100	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 2 as intermediary	21			1	2	1							
	22						3	100	0	False			
	21			1	2	2							
	22						3	0	3	False			
	21			1	2	3							
	22						3	1	9	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 100 & 0 & 3 & 4 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 0 & - & 2 & 2 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	20 – 21			2	2	0							
	22						0	100	100	False			
	21			2	2	1							
	22						0	100	100	False			
	21			2	2	2							
	22						0	0	0	False			
	21			2	2	3							
	22						0	1	1	False			
	20 – 21			3	2	0							
	22						9	100	1	False			
	21			3	2	1							
	22						9	100	6	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 2 as intermediary	21			3	2	2							
	22						9	0	9	False			
	21			3	2	3							
	21						9	1	0	False			
Node 3 as intermediary	19 – 21			0	3	0							
	22						14	1	0	False			
	21			0	3	1							
	22						14	6	5	False			
	21			0	3	2							
	22						14	9	8	False			
	21			0	3	3							
	22						14	0	14	False			
	20 – 21			1	3	0							
	22						4	1	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 100 & 100 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 2 & - & 2 & 2 \\ 0 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			1	3	1							
	22						4	6	0	False			
	21			1	3	2							
	22						4	9	3	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 3 as intermediary	21			1	3	3							
	22						4	0	4	False			
	20 – 21			2	3	0							
	22						1	1	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 2 & 100 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 2 & - & 2 & 2 \\ 3 & 1 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			2	3	1							
	22						1	6	100	True			
	23 – 24	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 2 & 7 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 2 & - & 2 & 2 \\ 3 & 3 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$										
	21			2	3	2							
	22						1	9	0	False			
	21			2	3	3							
	22						1	0	1	False			
	20 – 21			3	3	0							
	22						0	1	1	False			
	21			3	3	1							
	22						0	6	6	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 3 as intermediary	21			3	3	2							
	22						0	9	9	False			
	21			3	3	3							
	22						0	0	0	False			
Invalid path check	31 – 37	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 2 & 7 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 2 & - & 2 & 2 \\ 3 & 3 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$								3	1	
Shortest path retrieval	40 – 51	$\begin{bmatrix} 0 & 5 & 8 & 9 \\ 5 & 0 & 3 & 4 \\ 2 & 7 & 0 & 1 \\ 1 & 6 & 9 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 1 & 1 \\ 2 & - & 2 & 2 \\ 3 & 3 & - & 3 \\ 0 & 0 & 0 & - \end{bmatrix}$								3	1	**Distance from 3 to 1 is: 6** **The shortest path is: 3 -> 0 -> 1**

Appendix E

Trace Table for Best Case Scenario of Floyd-Warshall Algorithm

The trace table below uses `i`, `j`, and `k` to represent the row index, column index, and intermediary node respectively, which correspond to the variables `row`, `col`, and `middle` in code (see Figure 11 for code implementation and Figure 18 for the graph used for trace table).

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Data initiation	-	$\begin{bmatrix}0&1&1\\1&0&1\\1&1&0\end{bmatrix}$									0	2	
Next matrix initialization	7 - 16	$\begin{bmatrix}0&1&1\\1&0&1\\1&1&0\end{bmatrix}$	$\begin{bmatrix}-&1&2\\0&-&2\\0&1&-\end{bmatrix}$										
Node 0 as intermediary	19 – 21			0	0	0							
	22						0	0	0	False			
	21			0	0	1							
	22						0	1	1	False			
	21			0	0	2							
	22						0	1	1	False			
	20 – 21			1	0	0							
	22						1	0	1	False			
	21			1	0	1							
	22						1	1	0	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 0 as intermediary	21			1	0	2							
	22						1	1	1	False			
	20 – 21			2	0	0							
	22						1	0	1	False			
	21			2	0	1							
	22						1	1	1	False			
	21			2	0	2							
	22						1	1	0	False			
Node 1 as intermediary	19 – 21			0	1	0							
	22						1	1	0	False			
	21			0	1	1							
	22						1	0	1	False			
	21			0	1	2							
	22						1	1	1	False			
	20 – 21			1	1	0							
	22						0	1	1	False			
	21			1	1	1							
	22						0	0	0	False			
	21			1	1	2							
	22						0	1	1	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 1 as intermediary	20 – 21			2	1	0							
	22						1	1	1	False			
	21			2	1	1							
	22						1	0	1	False			
	21			2	1	2							
	22						1	1	0	False			
Node 2 as intermediary	19 – 21			0	2	0							
	22						1	1	0	False			
	21			0	2	1							
	22						1	1	1	False			
	21			0	2	2							
	22						1	0	1	False			
	20 – 21			1	2	0							
	22						1	1	1	False			
	21			1	2	1							
	22						1	1	0	False			
	21			1	2	2							
	22						1	0	1	False			
	20 – 21			2	2	0							
	22						0	1	1	False			

Stage	Line Number	distMatrix	nextMatrix	i	k	j	distMatrix[i][k]	distMatrix[k][j]	distMatrx[i][j]	Traverse through intermediary node yields shorter path?	start	end	Output
Node 2 as intermediary	21			2	2	1							
	22						0	1	1	False			
	21			2	2	2							
	22						0	0	0	False			
Invalid path check	31 – 37	$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 2 \\ 0 & - & 2 \\ 0 & 1 & - \end{bmatrix}$								0	2	
Shortest path retrieval	40 – 51	$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} - & 1 & 2 \\ 0 & - & 2 \\ 0 & 1 & - \end{bmatrix}$								0	2	**Distance from 0 to 2 is: 1** **The shortest path is: 0 -> 2**

Appendix F

Modified Code Snippet for Tracking Execution Time

Main.java

```
public class Main {

    private record GraphInList(Graph graph, Graph.Vertex start, Graph.Vertex end) {}
    private record GraphInMatrix(double[][] matrix, ArrayList<Graph.Vertex> vertexList, Graph.Vertex start, Graph.Vertex end) {}

    private static final ArrayList<GraphInList> adjacency_list_array = new ArrayList<>();
    private static final ArrayList<GraphInMatrix> adjacency_matrix_array = new ArrayList<>();

    private static final ArrayList<Long> bellmanFordTime = new ArrayList<>();
    private static final ArrayList<Long> floydWarshallTime = new ArrayList<>();

    public static void main(String[] args) {

        String example_input = """
        [(A,B,2); (A,C,3); (B,C,1); (C,D,4); (D,A,2); (B,D,3)] | A | D
        [(A,B,1); (A,C,4); (A,D,2); (B,C,2); (B,E,3); (C,D,1); (D,E,2); (E,A,5)] | A | E
        [(A,B,2); (A,C,5); (B,C,1); (B,D,3); (C,D,2); (C,E,4); (D,E,1); (E,F,3); (F,A,2); (D,F,5)] | A | F
        [(A,B,3); (A,C,2); (A,D,4); (B,C,1); (B,E,5); (C,D,3); (C,F,6); (D,E,2); (E,F,1); (F,A,4); (D,F,2)] | A | F
        """;
        preprocessing_steps(example_input);

        int numOfGraphs = adjacency_list_array.size();
        int iterationCount = 1000;
        for (int iter = 0; iter < iterationCount; iter++) {
            for (int i = 0; i < numOfGraphs; i++) {

                System.out.println("--- Bellman-Ford: Graph " + (i + 1) + ": ---");
                System.out.println(Bellman_Ford.getShortestPath(
                        adjacency_list_array.get(i).graph,
                        adjacency_list_array.get(i).start,
                        adjacency_list_array.get(i).end
                ));
                System.out.println();

                System.out.println("--- Floyd-Warshall: Graph " + (i + 1) + ": ---");
                System.out.println(Floyd_Warshall.getShortestPath(
                        adjacency_matrix_array.get(i).matrix,
                        adjacency_matrix_array.get(i).vertexList,
                        adjacency_matrix_array.get(i).start,
                        adjacency_matrix_array.get(i).end
                ));
                System.out.println();
            }
        }

        double bellmanFordAvg = bellmanFordTime.stream()
                .mapToDouble(d -> d)
                .average()
                .orElse(0.0);
```

```
        double floydWarshallAvg = floydWarshallTime.stream()
                .mapToDouble(d -> d)
                .average()
                .orElse(0.0);

        System.out.println("Bellman-Ford Time (nanoseconds): " + bellmanFordAvg);
        System.out.println("Floyd-Warshall Time (nanoseconds): " + floydWarshallAvg);

        try {
            PrintWriter writer = new PrintWriter("data.csv");
            writer.println("Run,Edge Num,Vertex Num,Bellman-Ford,Floyd-Warshall");
            for (int i = 0; i < numOfGraphs * iterationCount; i++) {
                writer.println((i + 1) + "," +
                        adjacency_list_array.get(i % 4).graph.getEdges().size() + "," +
                        adjacency_list_array.get(i % 4).graph.getSortedVertices().size() + "," +
                        bellmanFordTime.get(i) + "," +
                        floydWarshallTime.get(i));
            }
            writer.close();
        } catch (FileNotFoundException _) {}
    }

    public static void preprocessing_steps(String input_data) {

        String[] graphs_string = input_data.split("\n");

        for (String graph_string: graphs_string) {

            String[] split_array = graph_string.split("\\s*\\|\\s*");
            String graph_in_string = split_array[0];
            String start_in_string = split_array[1];
            String end_in_string = split_array[2];

            Graph graph = new Graph(graph_in_string);

            Graph.Vertex start_vertex = Graph.Vertex.find(start_in_string);
            Graph.Vertex end_vertex = Graph.Vertex.find(end_in_string);

            GraphInList list_result = new GraphInList(graph, start_vertex, end_vertex);
            adjacency_list_array.add(list_result);

            GraphInMatrix matrix_result = new GraphInMatrix(graph.convertToMatrix(), graph.getSortedVertices(), start_vertex, end_vertex);
            adjacency_matrix_array.add(matrix_result);
        }
    }

    // Function to assist in time recording for Bellman Ford algorithm
    public static void bellmanRecord(long start, long end) {
        bellmanFordTime.add(end - start);
    }

    // Function to assist in time recording for Floyd Warshall algorithm
    public static void floydRecord(long start, long end) {
        floydWarshallTime.add(end - start);
    }
}
```

Bellman_Ford.java

```
public class Bellman_Ford {

    public static String getShortestPath(Graph graph, Graph.Vertex start, Graph.Vertex destination) {

        ArrayList<Graph.Vertex> vertices = graph.getSortedVertices();
        int verticesCount = vertices.size();

        if (start.equals(destination)) {
            return "Distance from " + start + " to " + destination + " is: 0";
        }

        Map<Graph.Vertex, Integer> indexOf = new HashMap<>();
        for (int i = 0; i < verticesCount; i++) {
            indexOf.put(vertices.get(i), i);
        }

        Integer startIdx = indexOf.get(start);
        Integer endIdx = indexOf.get(destination);

        if (startIdx == null || endIdx == null) {
            return "Error. No path is found.";
        }

        double[] dist = new double[verticesCount];
        Arrays.fill(dist, Double.POSITIVE_INFINITY);
        int[] prev = new int[verticesCount];
        Arrays.fill(prev, -1);

        dist[startIdx] = 0.0;

        ArrayList<EdgeIndex> edges = new ArrayList<>();
        for (Graph.Edge edge : graph.getEdges()) {

            Integer startNode = indexOf.get(edge.startNode);
            Integer endNode = indexOf.get(edge.endNode);

            if (startNode != null && endNode != null) {
                edges.add(new EdgeIndex(startNode, endNode, edge.weight));
            }
        }

        // Record time
        long startTime = System.nanoTime();

        // Algorithm starts here
        for (int i = 0; i < verticesCount - 1; i++) {

            for (EdgeIndex edge : edges) {
                if (dist[edge.startIndex] == Double.POSITIVE_INFINITY) continue;
                double newDist = dist[edge.startIndex] + edge.weight;
                if (newDist < dist[edge.endIndex]) {
                    dist[edge.endIndex] = newDist;
                    prev[edge.endIndex] = edge.startIndex;
                }
            }
        }
```

```
        // Record ending time
        Main.bellmanRecord(startTime, System.nanoTime());

        for (EdgeIndex e : edges) {
            if (dist[e.startIndex] == Double.POSITIVE_INFINITY) continue;
            if (dist[e.startIndex] + e.weight < dist[e.endIndex]) {
                return "Negative cycle found from vertex " + start + ". This path is invalid.";
            }
        }

        double shortest = dist[endIdx];
        if (shortest == Double.POSITIVE_INFINITY) {
            return "Distance from " + start + " to " + destination + " is: Not found";
        }

        ArrayList<String> path = new ArrayList<>();
        int current = endIdx;
        while (current != -1) {
            path.addFirst(vertices.get(current).label());
            if (current == startIdx) break;
            current = prev[current];
        }

        if (path.isEmpty() || !path.getFirst().equals(start.label())) {
            return "Error. No path is found.";
        }

        return "Distance from " + start + " to " + destination + " is: " + shortest +
                "\nThe shortest path is: " + String.join(" -> ", path);
    }

    private record EdgeIndex(int startIndex, int endIndex, double weight) {}
}
```

`Floyd_Warshall.java`

```
public class Floyd_Warshall {

    public static String getShortestPath(double[][] distMatrix,
                                         ArrayList<Graph.Vertex> vertexList,
                                         Graph.Vertex start,
                                         Graph.Vertex end) {

        int matrixSize = vertexList.size();
        Integer[][] nextMatrix = new Integer[matrixSize][matrixSize];

        for (int row = 0; row < matrixSize; row++) {
            for (int col = 0; col < matrixSize; col++) {
                if (row != col && distMatrix[row][col] < Double.POSITIVE_INFINITY) {
                    nextMatrix[row][col] = col;
                }
            }
        }
```

```
        // Record time
        long startTime = System.nanoTime();

        // Algorithm starts here
        for (int middle = 0; middle < matrixSize; middle++) {
            for (int row = 0; row < matrixSize; row++) {
                for (int col = 0; col < matrixSize; col++) {
                    if (distMatrix[row][middle] + distMatrix[middle][col] < distMatrix[row][col]) {
                        distMatrix[row][col] = distMatrix[row][middle] + distMatrix[middle][col];
                        nextMatrix[row][col] = nextMatrix[row][middle];
                    }
                }
            }
        }

        // Record ending time
        Main.floydRecord(startTime, System.nanoTime());

        int startNodeIndex = vertexList.indexOf(start);
        int endNodeIndex = vertexList.indexOf(end);

        if (startNodeIndex == endNodeIndex)
            return "Distance from " + start + " to " + end + " is: 0";

        try {
            double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];
            if (shortestDistance == Double.POSITIVE_INFINITY) throw new ArrayIndexOutOfBoundsException();
        } catch (ArrayIndexOutOfBoundsException _) {
            return "Distance from " + start + " to " + end + " is: Not found";
        }

        double shortestDistance = distMatrix[startNodeIndex][endNodeIndex];

        ArrayList<String> path = new ArrayList<>();
        path.add(start.label());

        int currentNodeIndex = startNodeIndex;
        while (currentNodeIndex != endNodeIndex) {

            Integer middleVertex = nextMatrix[currentNodeIndex][endNodeIndex];

            if (middleVertex == null)
                return "Error. No path is found.";

            path.add(vertexList.get(middleVertex).label());
            currentNodeIndex = middleVertex;
        }

        return "Distance from " + start + " to " + end + " is: " + shortestDistance +
                "\nThe shortest path is: " + String.join(" -> ", path);
    }
}
```

The sample sparse graphs used are:

```
String example_input = """
[(A, B, 2); (B, C, 3); (C, D, 4)] | A | D
[(A, B, 1); (A, C, 5); (C, D, 2)] | A | D
[(A, B, 2); (B, C, 2); (C, D, 2); (D, E, 2)] | A | E
[(A, B, 3); (A, C, 8); (C, D, 1); (D, E, 1)] | A | E
[(A, B, 1); (B, C, 2); (C, D, 3); (D, E, 4); (E, F, 5)] | A | F
[(A, B, 4); (B, D, 1); (D, F, 2); (A, C, 10); (C, E, 3)] | A | F
[(A, B, 2); (B, C, 2); (C, D, 2); (D, E, 2); (E, F, 2); (F, G, 2)] | A | G
[(A, B, 5); (A, C, 9); (B, D, 3); (C, E, 4); (D, F, 6); (E, G, 1)] | A | G
[(A, B, 1); (B, C, 1); (C, D, 1); (D, E, 1); (E, F, 1); (F, G, 1); (G, H, 1)] | A | H
[(A, B, 3); (A, C, 7); (B, D, 2); (C, E, 6); (D, F, 4); (E, G, 5); (F, H, 2)] | A | H
""";
```

with the average time execution based on the number of vertices as follows:

Number of Vertices	Bellman-Ford (in ns)	Floyd-Warshall (in ns)
3	1016.98046	1450.68227
4	997.997727	1467.97273
5	976.79	1565.65409
6	1078.73	1456.15591
7	1139.25909	1550.04273
8	1012.01455	1541.44

The sample dense graphs used are:

```
String example_input = """
[(A,B,2); (A,C,3); (B,A,2); (B,C,1); (C,A,3); (C,B,1)] | A | C
[(A,B,5); (A,C,2); (B,A,5); (B,C,4); (C,A,2); (C,B,4)] | A | B
[(A,B,2); (A,C,3); (A,D,4); (B,A,2); (B,C,1); (B,D,5); (C,A,3); (C,B,1); (C,D,2); (D,A,4); (D,B,5); (D,C,2)] | A | D
[(A,B,1); (A,C,6); (A,D,3); (B,A,1); (B,C,2); (B,D,4); (C,A,6); (C,B,2); (C,D,1); (D,A,3); (D,B,4); (D,C,1)] | A | D
[(A,B,4); (A,C,2); (A,D,5); (B,A,4); (B,C,3); (B,D,1); (C,A,2); (C,B,3); (C,D,6); (D,A,5); (D,B,1); (D,C,6)] | A | D
[(A,B,1); (A,C,4); (A,D,7); (A,E,3); (B,A,1); (B,C,2); (B,D,6); (B,E,5); (C,A,4); (C,B,2); (C,D,1); (C,E,3); (D,A,7); (D,B,6); (D,C,1); (D,E,2); (E,A,3); (E,B,5); (E,C,3); (E,D,2)] | A | E
[(A,B,3); (A,C,2); (A,D,6); (A,E,4); (B,A,3); (B,C,1); (B,D,5); (B,E,2); (C,A,2); (C,B,1); (C,D,4); (C,E,3); (D,A,6); (D,B,5); (D,C,4); (D,E,1); (E,A,4); (E,B,2); (E,C,3); (E,D,1)] | A | E
[(A,B,2); (A,C,5); (A,D,1); (A,E,6); (A,F,3); (B,A,2); (B,C,4); (B,D,7); (B,E,2); (B,F,5); (C,A,5); (C,B,4); (C,D,1); (C,E,3); (C,F,6); (D,A,1); (D,B,7); (D,C,1); (D,E,4); (D,F,2); (E,A,6); (E,B,2); (E,C,3); (E,D,4); (E,F,1); (F,A,3); (F,B,5); (F,C,6); (F,D,2); (F,E,1)] | A | F
[(A,B,4); (A,C,2); (A,D,6); (A,E,3); (A,F,5); (B,A,4); (B,C,3); (B,D,2); (B,E,6); (B,F,1); (C,A,2); (C,B,3); (C,D,4); (C,E,5); (C,F,6); (D,A,6); (D,B,2); (D,C,4); (D,E,1); (D,F,3); (E,A,3); (E,B,6); (E,C,5); (E,D,1); (E,F,2); (F,A,5); (F,B,1); (F,C,6); (F,D,3); (F,E,2)] | A | F
[(A,B,1); (A,C,5); (A,D,2); (A,E,4); (A,F,3); (B,A,1); (B,C,2); (B,D,6); (B,E,5); (B,F,4); (C,A,5); (C,B,2); (C,D,1); (C,E,3); (C,F,2); (D,A,2); (D,B,6); (D,C,1); (D,E,4); (D,F,5); (E,A,4); (E,B,5); (E,C,3); (E,D,4); (E,F,1); (F,A,3); (F,B,4); (F,C,2); (F,D,5); (F,E,1)] | A | F
""";
```

with the average time execution based on the number of vertices as follows:

Number of Vertices	Bellman-Ford (in ns)	Floyd-Warshall (in ns)
3	251.1115	195.559
4	539.478667	468.305
5	1109.3025	707.145
6	1838.26967	1174.32233